Operator-Symbolic Languages for Artificial Intelligence

Geometry, Proof-Carrying Semantics, and Shared Algebraic Communication

Komorebi P.

Finite metatheory and independent-kernel draft v0.5

Abstract

We develop an operator-symbolic language as a candidate machine-native intermediate representation for artificial intelligence and introduce OSL-Core, a concrete executable subset. The central thesis is that continuous latent geometry, typed operators, symbolic relations, effect annotations, proof objects, and provenance can be combined into one language in which states function as nouns, operators as verbs, types as grammatical categories, logic as admissibility syntax, causal consequences as semantics, and proof paths as explanations. Distinct models need not share coordinates: they may share an abstract transformation algebra realized through different latent representations and connected by exact or approximate intertwiners. We define the language family $\mathbb{L}_{\mathrm{AI}}$, give denotational, dynamic, graded, coalgebraic, and effect semantics, and then fix a concrete surface grammar with agents, lineages, vector spaces, projectors, coordinate transport, provenance-aware packets, communication, quarantine, endorsement, memory, planning, capabilities, feedback, assertions, and canonical receipts. Version 0.3 adds linear and affine proof resources, append-only audit semantics, statement-boundary receipts, deterministic semantic replay, finite session-graph verification, explicit-flow analysis, an independent receipt kernel, and two exhaustive finite transition-model layers. The packaged evidence comprises sixty-six source programs, two paired public-projection audits, sixty-one focused tests, 1,149 independently reconstructed receipts, seventy proof terms, 2,330 exhaustively reached abstract states, and 47,958 finite transition checks. Within a finite-dimensional base theory we prove preservation of polynomial operator semantics, approximate relation bounds, coordinate invariance, finite recovery of observational congruences, quotient emergence of symbols, geometric realization of an ordered logical fragment, stability of graded formulas, content-only ownership impossibility, proof-gated execution safety, proof transport, packet block identities, exact interlingua factorization, private closure under channel cuts, finite metadata-capacity bounds, tensor-product preservation, finite-horizon nonlinear transport, effect confinement, and finite-core provenance non-forging. The new computer-assisted results establish the declared finite properties of the abstract kernels and receipt semantics, but do not constitute a proof-assistant verification of the complete interpreter. No claim concerning subjective experience is made.

Draft status

Version 0.5 consolidates the concrete language as OSL-Core 0.3. It preserves the typed AST, proof-gated memory and planning, bounded capabilities, contracts, rollback, sessions, neural proposal quarantine, and metadata audits of version 0.2, while adding resource-sensitive proofs, an append-only audit chain, semantic source-boundary receipts, exact replay certificates, verified finite session graphs, static explicit-flow certificates, an independent receipt/proof kernel, and two complete finite transition-model explorations. Sixty-six source programs and sixty-one focused tests pass their declared specifications. Exact finite results are stated with their model and trust boundaries; approximate and nonlinear claims retain error bounds

or open-theory labels. A proof-assistant formalization of the complete syntax and runtime remains open.

Claim boundary

The framework concerns representations, transformations, proof, provenance, communication, and control. It does not establish consciousness, sentience, personhood, valence, or moral status. A system may implement every formal construction in this paper without possessing subjective experience.

Contents

1 Introduction **8**
1.1 The language problem for artificial agents . . . 8
1.2 States, operators, and proof paths . . . 8
1.3 Why natural language is not enough . . . 8
1.4 Main contributions . . . 9

2 Architectural overview **9**
2.1 Four representational layers . . . 9
2.2 The hybrid thesis . . . 10
2.3 Design axioms . . . 10

3 Mathematical preliminaries **11**
3.1 Typed spaces and maps . . . 11
3.2 Operator algebras . . . 11
3.3 Programs and transitions . . . 12
3.4 Proof objects . . . 12

4 Definition of the AI-native language **12**
4.1 Core types . . . 12
4.2 Terms . . . 13
4.3 Formulas . . . 13
4.4 Programs . . . 13
4.5 Proof terms . . . 13
4.6 Effect judgments . . . 13

5 OSL-Core: a concrete operator-symbolic language **14**
5.1 Design goals . . . 14
5.2 Lexical conventions . . . 15
5.3 Concrete type grammar . . . 15
5.4 Provenance classes . . . 16
5.5 Resource qualifiers . . . 16
5.6 Effect rows and capabilities . . . 17
5.7 Surface grammar . . . 17
5.8 Declaration forms . . . 20
5.9 Operator forms . . . 20

5.10 Packet constructors . . . 20
5.11 Communication . . . 21
5.12 Binding, endorsement, and authorization . . . 21
5.13 Memory writes . . . 21
5.14 Planning and execution . . . 21
5.15 Feedback . . . 22
5.16 Assertions and executable specification . . . 22
5.17 Canonical packet representation . . . 22

6 From a source statement to proof-carrying execution 23
6.1 Compiled packet trace . . . 23

7 Worked OSL-Core programs 24
7.1 Example inventory . . . 24
7.2 Direct observation and typed autobiography . . . 24
7.3 Communicated evidence and action . . . 25
7.4 Activation injection and quarantine . . . 25
7.5 Dream content and simulation provenance . . . 26
7.6 Projector declaration and application . . . 26
7.7 Coordinate transport . . . 27
7.8 Exact semantic intertwiner . . . 27
7.9 Fork lineage . . . 28
7.10 Planner binding and capability gating . . . 28
7.11 Inference is not observation . . . 28
7.12 Foreign content and autobiographical safety . . . 29
7.13 Consequence feedback . . . 29
7.14 Three-agent shared workspace . . . 30
7.15 Provisional reconstruction . . . 30
7.16 Tool-call effect . . . 31
7.17 Negative examples as language tests . . . 31
7.18 Source-to-IR lowering example . . . 32
7.19 A tensor-valued operator example . . . 33
7.20 A nonlinear repair example . . . 33
7.21 Example-suite result . . . 34

8 Static and operational semantics of OSL-Core 34
8.1 Static contexts . . . 34
8.2 Runtime configuration . . . 35
8.3 Observation rule . . . 35
8.4 Communication rule . . . 35
8.5 Injection and quarantine . . . 36
8.6 Endorsement rule . . . 36
8.7 Authorization rule . . . 36
8.8 Memory-write rule . . . 36
8.9 Planning rule . . . 36
8.10 Execution rule . . . 37
8.11 Simulation isolation . . . 37
8.12 Numeric operator semantics . . . 37

8.13 Canonical receipts . . . 37
8.14 Metatheoretic properties of the finite core . . . 38
8.15 Reference interpreter . . . 39

9 OSL-Core 0.2: mechanized finite control layer 39
9.1 Explicit abstract syntax and static checking . . . 40
9.2 First-class proof terms . . . 40
9.3 Proof-carrying planning example . . . 41
9.4 Bounded and expiring capabilities . . . 42
9.5 Temporal contracts . . . 43
9.6 Transactional updates and rollback . . . 43
9.7 Finite-state shared-workspace sessions . . . 44
9.8 Neural boundary proposals . . . 45
9.9 Paired-trace noninterference audit . . . 46
9.10 Mechanized finite guarantees and their limits . . . 47
9.11 Expanded implementation map . . . 47
9.12 Next metatheoretic step . . . 48

10 OSL-Core 0.3: finite metatheory and an independent audit kernel 48
10.1 Resource-sensitive proof terms . . . 49
10.2 Source-bound append-only audit semantics . . . 50
10.3 Independent receipt and proof reconstruction . . . 51
10.4 Complete finite abstract kernel . . . 52
10.5 Decomposed finite metacore . . . 53
10.6 Verified finite session graphs . . . 54
10.7 Static and dynamic information-flow certificates . . . 55
10.8 Integrated verification totals . . . 55
10.9 What the computer-assisted results do and do not prove . . . 56
10.10 Reproducible verification pipeline . . . 57

11 Denotational semantics 57
11.1 Types and terms . . . 57
11.2 Formula semantics . . . 57
11.3 Graded semantics . . . 57
11.4 Dynamic semantics . . . 58
11.5 Causal semantics . . . 58
11.6 Coalgebraic trace semantics . . . 58

12 Meaning as shared algebra rather than shared coordinates 59
12.1 Heterogeneous internal spaces . . . 59
12.2 Approximate intertwiners . . . 59
12.3 Gauge equivalence . . . 60

13 Symbols as compressed operator invariants 61
13.1 Functional equivalence . . . 61
13.2 Developmental sequence . . . 61
13.3 Finite observational recovery . . . 62

14 Logical structure realized geometrically **63**
14.1 Candidate geometries . . . 63
14.2 Implication cones . . . 63
14.3 Negation . . . 63
14.4 Conjunction . . . 63
14.5 A finite ordered geometric fragment . . . 63

15 Probabilistic and paraconsistent semantics **64**
15.1 Four-valued support . . . 64
15.2 No explosion . . . 65
15.3 Quantitative semantic stability . . . 65
15.4 Confidence as a first-class term . . . 65
15.5 Decision rules . . . 65

16 Provenance and proof-carrying execution **66**
16.1 Provenance records . . . 66
16.2 Content is not ownership . . . 66
16.3 Proof-gated actions . . . 66
16.4 Linear provenance resources . . . 66
16.5 Proof transport between calculi . . . 67

17 Shared operator-symbolic languages **67**
17.1 Common interlingua . . . 67
17.2 Shared packet . . . 67
17.3 Functional transport objective . . . 67
17.4 Interlingua factorization . . . 68
17.5 Communication is not merger . . . 68
17.6 Interface-decomposable communication . . . 68
17.7 Group-level task irreducibility . . . 69
17.8 Canonical metadata and covert capacity . . . 69

18 Komorebi-style operator grammar **70**
18.1 Analysis, packets, transfer, synthesis . . . 70
18.2 Exact packet block algebra . . . 70
18.3 Residuals . . . 71
18.4 Holonomy . . . 71
18.5 Multiresolution representation . . . 71

19 Interaction-induced latent geometry **71**
19.1 Transient activation geometry . . . 71
19.2 Persistent weight geometry . . . 71
19.3 Training objective . . . 72
19.4 From interaction relations to an internal representation . . . 72

20 Tensor and nonlinear generalizations **72**
20.1 Tensor-valued states . . . 72
20.2 Nonlinear retractions . . . 73
20.3 Nonlinear semantics . . . 73
20.4 Coordinate covariance . . . 73

20.5 Koopman and kernel lifts . . . 73
20.6 Finite-horizon approximate conjugacy . . . 73

21 A compiler architecture **74**
21.1 Front end . . . 74
21.2 Intermediate representation . . . 74
21.3 Verifier . . . 75
21.4 Effect confinement . . . 75
21.5 Back end . . . 75

22 Assumption ledger **75**

23 Formal theorem program **77**
23.1 Results established here . . . 77
23.2 Remaining theorem targets . . . 77

24 Experimental program **78**
24.1 Experiment A: operator algebra induction . . . 78
24.2 Experiment B: symbol emergence . . . 78
24.3 Experiment C: heterogeneous-agent communication . . . 78
24.4 Experiment D: provenance-sensitive communication . . . 78
24.5 Experiment E: proof-gated planning . . . 78
24.6 Experiment F: context-to-weight consolidation . . . 78
24.7 Experiment G: nonlinear transfer . . . 79
24.8 Evaluation standards . . . 79
24.9 Essential ablations . . . 79

25 Falsification criteria **79**

26 Limitations and boundaries **80**
26.1 No unique internal language . . . 80
26.2 Interpretability remains incomplete . . . 80
26.3 Proof verifier assumptions . . . 80
26.4 Optimization can find shortcuts . . . 80
26.5 Natural language remains useful . . . 80
26.6 No consciousness claim . . . 81

27 Discussion **81**
27.1 Language as transformation algebra . . . 81
27.2 A cognitive intermediate representation . . . 81
27.3 Machine-native communication . . . 81
27.4 Relation to persistent agency . . . 81
27.5 What the theory predicts about learned representations . . . 81
27.6 Why operators and logic are complementary . . . 81

28 Conclusion **82**

A Notation **82**

B Typing rules **83**

C A minimal packet schema **83**

D Reference implementation modules **83**
D.1 Executable finite kernel 84
D.2 Proposed full implementation 84

E Theorem and dependency status **85**

F Open theory modules **86**

G Selected references and bibliographic audit boundary **87**

1 Introduction

1.1 The language problem for artificial agents

Artificial agents already manipulate high-dimensional internal states, transform them through learned operators, exchange messages, update memories, and select actions. Yet the representational levels used for these functions are often separated:

- neural activations carry continuous information but are difficult to inspect;
- symbolic structures are inspectable but often brittle under noise and novel inputs;
- natural language is expressive but ambiguous and weakly typed;
- action policies are executable but usually do not carry their own provenance or proof.

We ask whether these layers can be unified by a machine-native intermediate language whose primitive objects are not merely words, propositions, or vectors, but *typed transformations of structured states*. The motivating slogan is:

$$\boxed{\begin{aligned}&\text{continuous geometry} \longrightarrow \text{typed operator algebra}\\ &\qquad\qquad\qquad\longrightarrow \text{symbolic logic and proof} \longrightarrow \text{language or action.}\end{aligned}} \tag{1}$$

The proposal does not require neural geometry to disappear. Instead, symbolic expressions name and constrain reusable operator patterns discovered in continuous representations.

1.2 States, operators, and proof paths

A latent state is a candidate noun. An operator is a candidate verb. A type specifies the admissible input and output roles. A formula expresses a relation that must hold. A proof term certifies that an inference or operation is authorized. A causal consequence supplies semantics by testing whether the transformation behaves as predicted.

The intended interpretation is summarized by:

Language-like role	Computational realization
Noun	state, packet, memory, object, distribution
Verb	operator, transition, transport, update
Type	admissible role or interface
Syntax	composition and logical well-formedness
Semantics	causal effect and denotation
Authorship	provenance and lineage
Explanation	proof path and receipt chain
Discourse through time	recurrence and memory update

1.3 Why natural language is not enough

Natural language is valuable as a human interface, but it is usually:

- ambiguous;
- context-sensitive;

- difficult to execute directly;
- weak at representing provenance and authorization;
- difficult to verify mechanically;
- vulnerable to hidden or unintended side channels.

An operator-symbolic intermediate representation can instead be typed, compositional, executable, uncertainty-aware, provenance-preserving, and machine-checkable.

1.4 Main contributions

The paper develops the following components:

1. a typed operator-symbolic language $\mathbb{L}_{\mathrm{AI}}$ with terms, formulas, programs, proof terms, and effects;
2. denotational, operational, graded, causal, and effect semantics over latent state spaces;
3. a coordinate-free notion of meaning through operator algebras, presentations, and intertwiners;
4. finite observational recovery of operator congruences and quotient symbols;
5. geometric realizations of order, negation, and conjunction without assuming Boolean collapse;
6. provenance-sensitive, resource-aware, and proof-carrying assertions and actions;
7. probabilistic and paraconsistent logical states with quantitative stability bounds;
8. proof translation between heterogeneous symbolic calculi under explicit rule-preservation conditions;
9. shared algebraic communication among heterogeneous models, with operational criteria for private versus group-level closure;
10. Komorebi-style packet analysis, transfer blocks, synthesis, residuals, and holonomy;
11. nonlinear, tensor, manifold, Koopman, kernel, and tensor-network extensions;
12. a compiler architecture with effect confinement and verification-condition generation;
13. explicit experiments, baseline families, falsification criteria, and theorem-status boundaries.

2 Architectural overview

2.1 Four representational layers

We distinguish four coupled layers.

Latent substrate. Continuous states:

$$h_t \in \mathcal{H}, \qquad x_t \in \mathcal{X},$$

may be vectors, tensors, graphs, probability distributions, neural activations, or manifold-valued states.

Operator layer. Typed maps act on states:

$$P_S x, \qquad P_J h, \qquad T_{A\to B} h, \qquad F(x,a), \qquad R(P_J h).$$

These operators perform selection, transport, prediction, reconstruction, and update.

Symbolic layer. Typed predicates and modalities express access, belief, provenance, ownership, authorization, and causal use.

Interface layer. Natural language, code, diagrams, plans, and physical actions render or execute the internal operator-symbolic state.

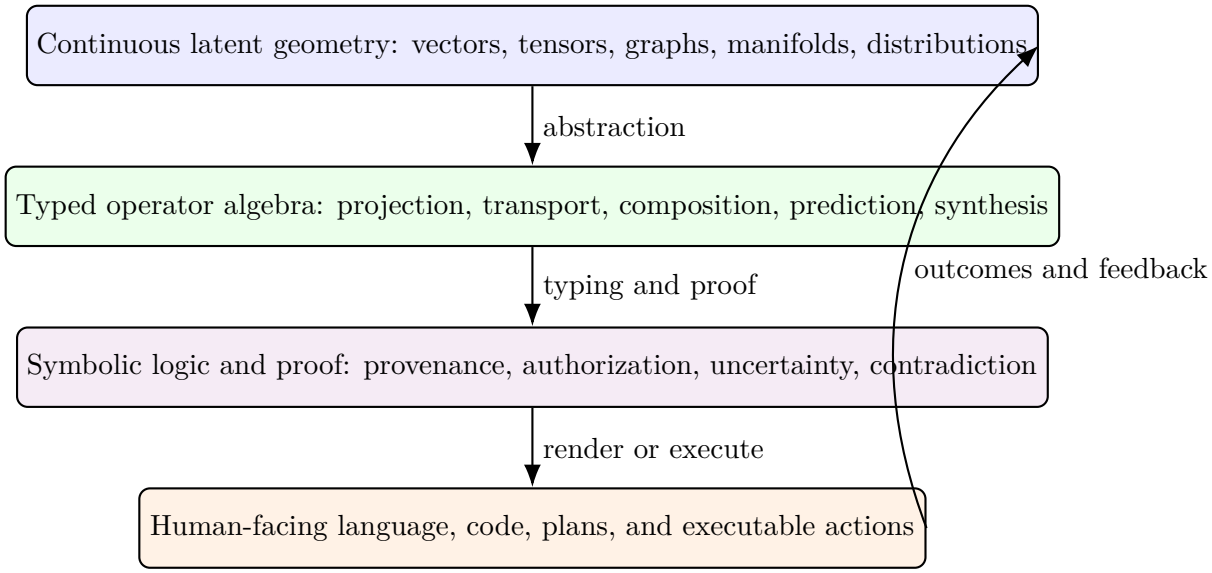


Figure 1: The proposed operator-symbolic stack. The feedback arrow is essential: causal consequences revise latent geometry, operators, and symbolic beliefs.

2.2 The hybrid thesis

The proposal is not that symbolic logic should replace representation learning. The intended architecture is:

$$\boxed{\text{latent learning} \leftrightarrow \text{operator intermediate representation} \leftrightarrow \text{symbolic proof layer.}} \tag{2}$$

The latent layer handles approximation and novelty. The operator layer makes transformations reusable and transportable. The symbolic layer constrains inference, memory, communication, and action.

2.3 Design axioms

The framework is organized around six design axioms.

A1. Gauge tolerance. Meaning is attached to operator relations and causal roles, not privileged coordinates.

A2. Compositionality. The semantics of a well-typed composite is determined by the semantics of its parts and their composition law.

A3. Causal grounding. Declared meanings must predict intervention-sensitive consequences, not merely correlate with labels.

A4. Proof-carrying use. Memory writes, tool calls, and high-impact actions require typed evidence satisfying declared preconditions.

A5. Provenance conservation. Communication may grant access to content without fabricating direct observation or ownership.

A6. Calibrated incompleteness. When available evidence does not identify one semantics, the language must preserve conflict or abstain rather than force a symbol.

These axioms deliberately separate representation learning from authorization. A learned latent state may propose an operator-symbolic interpretation; the verifier decides which consequences that interpretation is permitted to have.

3 Mathematical preliminaries

3.1 Typed spaces and maps

Let T be a collection of types. Each type $\tau \in \mathsf{T}$ is interpreted as a space:

$$[\![\tau]\!] = X_\tau.$$

For the finite base theory, each X_τ is a finite-dimensional real Hilbert space. More general interpretations are introduced later.

A typed operator symbol

$$g : \tau \to \sigma$$

is interpreted as a map

$$[\![g]\!] : X_\tau \to X_\sigma.$$

Composition is type checked:

$$g : \tau \to \sigma, \qquad f : \sigma \to \upsilon \quad \Longrightarrow \quad f \circ g : \tau \to \upsilon.$$

3.2 Operator algebras

Let $\mathcal{A}$ be an abstract algebra generated by primitive operations $\mathcal{G}$. An agent a realizes $\mathcal{A}$ through a representation

$$\rho_a : \mathcal{A} \to \mathrm{End}(\mathcal{H}_a).$$

When $\mathcal{A}$ is a monoid algebra, the representation satisfies

$$\rho_a(g_2 g_1) = \rho_a(g_2)\rho_a(g_1), \qquad \rho_a(e) = I.$$

Approximate representations permit bounded residuals.

3.3 Programs and transitions

A program α denotes a transition relation

$$[\![\alpha]\!] \subseteq \mathcal{S} \times \mathcal{S}$$

or, in deterministic settings, a map $[\![\alpha]\!] : \mathcal{S} \to \mathcal{S}$. Sequential composition satisfies

$$[\![\alpha; \beta]\!] = [\![\beta]\!] \circ [\![\alpha]\!].$$

3.4 Proof objects

A proof judgment has the form

$$\Gamma; \Pi \vdash \varphi,$$

where Γ is a type and term environment and Π is a provenance and authorization context. A proof term p inhabits a proposition type:

$$p : \varphi.$$

This Curry–Howard-style notation is used operationally: actions may require proof terms before execution.

4 Definition of the AI-native language

Definition 4.1 (Operator-symbolic AI language)**.** An operator-symbolic AI language is a tuple

$$\mathbb{L}_{\mathrm{AI}} = (\mathsf{Types}, \mathsf{Terms}, \mathsf{Operators}, \mathsf{Formulas}, \mathsf{Programs}, \mathsf{Proofs}, \mathsf{Effects}, \vdash, [\![\cdot]\!])$$

consisting of types, terms, typed operators, formulas, executable programs, proof terms, effect annotations, derivability rules, and semantics.

4.1 Core types

A minimal type system contains:

Type	Intended denotation	Example inhabitant
`State`	causal or latent state	x_t
`Workspace`	accessible packet state	j_t
`Agent`	agent identity or interface	a
`Lineage`	temporal or causal lineage	L_a
`Memory`	stored event or model	m
`Observation`	externally grounded event	o
`Simulation`	internally generated event	d
`Action`	executable intervention	u
`Proof`	authorization evidence	p
`Probability`	calibrated uncertainty	$q \in [0, 1]$
`Provenance`	source and origin status	ρ
`Receipt`	proof-carrying transition record	r
`Channel`	typed communication interface	χ
`Effect`	externally visible capability	$\mathsf{write}, \mathsf{act}$

4.2 Terms

Terms are generated by

$$t ::= x \mid c \mid g(t) \mid f(t_1, \ldots, t_n) \mid \mathrm{Source}(t) \mid \mathrm{Lineage}(t).$$

Representative operator terms include

$$P_S(t), \qquad P_J(t), \qquad T_{A\to B}(t), \qquad \mathrm{Memory}(t), \qquad \mathrm{Predict}(t, a).$$

4.3 Formulas

The formula grammar is

$$\begin{aligned}\varphi ::= {} & p(t_1, \ldots, t_n) \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \to \psi \\ & \mid J_a\varphi \mid B_{a,L}\varphi \mid O_a^{\kappa}\varphi \mid K_a^{q}\varphi \mid U_a\varphi \mid Q_a\varphi \mid [\alpha]\varphi \mid X\varphi.\end{aligned}$$

The modalities mean:

Expression	Meaning
$J_a\varphi$	φ is accessible to agent a
$B_{a,L}\varphi$	φ is bound to agent a under lineage L
$O_a^{\kappa}\varphi$	φ is autobiographically typed by provenance κ
$K_a^{q}\varphi$	a endorses φ with confidence q
$U_a\varphi$	φ is causally used in planning
$Q_a\varphi$	φ is quarantined
$[\alpha]\varphi$	after every execution of α, φ holds
$X\varphi$	φ holds in the next state

4.4 Programs

Programs are generated by

$$\begin{aligned}\alpha ::= {} & \mathrm{sense} \mid \mathrm{bind} \mid \mathrm{infer} \mid \mathrm{plan} \mid \mathrm{act} \mid \mathrm{update} \\ & \mid \mathrm{communicate}(\chi, t) \mid \mathrm{writeMemory}(t) \mid \mathrm{simulate}(\alpha) \mid \alpha; \beta \\ & \mid \mathrm{if}\ \ \varphi\ \ \mathrm{then}\, \alpha\ \ \mathrm{else}\, \beta.\end{aligned}$$

4.5 Proof terms

Proof terms may include:

$$p ::= \mathrm{obsToken}(e) \mid \mathrm{originToken}(a, e) \mid \mathrm{lineageToken}(L, e) \mid \mathrm{authToken}(\alpha) \mid \mathrm{compose}(p_1, p_2).$$

4.6 Effect judgments

A program judgment has the form

$$\Gamma; \Pi \vdash \alpha : \tau \rightsquigarrow \sigma \,!\, \mathcal{E},$$

where $\mathcal{E} \subseteq \mathsf{Effects}$ is the set of externally visible effects that the program may emit. Representative effects include

$$\mathsf{read}, \quad \mathsf{writeMemory}, \quad \mathsf{communicate}, \quad \mathsf{toolCall}, \quad \mathsf{externalAct}, \quad \mathsf{simulate}.$$

Sequential composition unions effects, while a capability proof may reduce an abstract requested effect to a narrower authorized effect.

5 OSL-Core: a concrete operator-symbolic language

The preceding definition describes a family of languages. This section fixes a small concrete member of that family, called OSL-Core. Its purpose is not to replace neural computation or to settle every metatheoretic question. It is a reference intermediate language in which operator declarations, provenance, proof obligations, memory writes, communication, effects, and action authorization can be stated in one typed program.

Draft status

The release accompanying this manuscript includes a deterministic reference interpreter for a finite subset, sixty-six source programs, sixty-one automated tests, and a machine-readable example-suite result. The interpreter implements the operational policies described below for finite arrays, provenance-aware packets, first-class proofs, bounded capabilities, temporal contracts, transactions, session protocols, neural proposals, memory writes, planning, effects, and receipts. It does not implement the complete higher-order, stochastic, nonlinear, or proof-assistant semantics proposed elsewhere in the paper.

5.1 Design goals

OSL-Core is designed around twelve requirements.

1. **Typed operator composition.** States and transformations carry explicit source and target types.
2. **Coordinate tolerance.** Meaning may be preserved by an intertwiner or transported operator even when concrete vectors differ.
3. **Provenance conservation.** Communication preserves source and lineage rather than converting foreign content into direct observation.
4. **Linear evidence resources.** Observation and authorization tokens cannot be silently forged or duplicated.
5. **Effect confinement.** External action, tool calls, memory writes, communication, simulation, and model modification are separate effects.
6. **Safe rejection.** A program that lacks a proof or capability is rejected or quarantined rather than implicitly coerced into validity.
7. **Canonical receipts.** Every state-changing instruction emits a canonical record whose digest binds the declared transition.
8. **Hybrid execution.** Learned latent modules may propose states and operators, while the typed verifier controls which proposals may become memory or action.
9. **First-class proofs.** Access, origin, clearance, endorsement, authorization, ownership, and use may be materialized as digest-bound proof terms rather than remaining implicit runtime flags.
10. **Temporal contracts.** Safety invariants and bounded response obligations are monitored over the execution trace.

11. **Transactional repair.** Memory and model updates may be checkpointed, committed, or rolled back atomically in the finite runtime.

12. **Protocol-typed sharing.** Finite-state sessions constrain the order and participants of shared-workspace communication.

5.2 Lexical conventions

Identifiers begin with a letter or underscore and may contain letters, digits, underscores, and hyphens. Comments begin with `#`. Strings are UTF-8 double-quoted literals. Numeric arrays use JSON-compatible bracket notation. Every source statement occupies one logical line in the reference subset, although a production parser would permit ordinary block layout.

The language is case-sensitive. Reserved words are shown in listing 1. Canonical serialization normalizes field order, numeric precision, set order, and whitespace before hashing; textual spelling is therefore not itself a trusted provenance source.

Listing 1: Reserved words in the finite reference subset.

```
module lineage extends agent grant effect
space dim vector in values operator kind matrix
apply transport check_projector check_intertwiner between and tolerance
observe infer simulate inject communicate from to as
bind quarantine endorse threshold authorize for write_memory
plan using expected execute feedback observed
proof mode persistent affine linear requires attest neural_propose admit
    reject_candidate
contract after within check_contracts begin_transaction commit_transaction
    rollback_transaction
session participants initial session_transition session_terminal verify_session
    start_session
secret metadata receipt_check verify_receipts
assert packet provenance accessible_to quarantined_by
memory contains action executed vector approx parent show
```

5.3 Concrete type grammar

The abstract type universe is refined to the following concrete family:

$$\begin{aligned}\tau ::= {} & \texttt{Bool} \mid \texttt{Nat} \mid \texttt{Real} \mid \texttt{String} \mid \texttt{Agent} \mid \texttt{Lineage} \\ & \mid \texttt{State}[n] \mid \texttt{Tensor}[d_1, \ldots, d_k] \mid \texttt{Operator}[\tau \Rightarrow \sigma] \\ & \mid \texttt{Packet}[\kappa, \tau] \mid \texttt{Workspace}[a, \tau] \mid \texttt{Memory}[a, \kappa, \tau] \\ & \mid \texttt{Proof}[\varphi] \mid \texttt{Action}[\mathcal{E}] \mid \texttt{Receipt}[\varphi] \mid \texttt{Distribution}[\tau].\end{aligned}$$

Here $n, d_i \in \mathbb{N}_{>0}$, a is an agent, κ is a provenance class, and $\mathcal{E}$ is an effect row.

Type	Static role	Runtime interpretation
`State[n]`	finite causal or latent state	vector in $\mathbb{R}^n$ in the reference backend
`Tensor[d1,...,dk]`	higher-order structured state	multidimensional array or tensor-network handle

Type	Static role	Runtime interpretation
`Operator[t=>s]`	typed transformation	matrix, nonlinear map, symbolic operator, or verified external primitive
`Packet[k,t]`	content with provenance class k	canonical content, source, lineage, confidence, authorization, digest
`Workspace[a,t]`	content accessible to agent a	packet admitted to the private or shared workspace
`Memory[a,k,t]`	autobiographical record of class k	immutable record linked to a source packet digest
`Proof[p]`	evidence for proposition p	proof term, capability, or verifier receipt
`Action[E]`	action whose effects are in E	planned transition awaiting capability-checked execution
`Receipt[p]`	canonical transition witness	integrity-bound record supporting proposition p

5.4 Provenance classes

The concrete provenance class is

$$\kappa \in \left\{ \begin{array}{l} \mathsf{observed}, \mathsf{inferred}, \mathsf{recalled}, \mathsf{simulated}, \mathsf{communicated}, \mathsf{injected}, \\ \mathsf{foreign}, \mathsf{reconstructed}, \mathsf{conflicted}, \mathsf{unresolved} \end{array} \right\}.$$

These tags are not mere comments. They constrain the constructors available for memory and planning. The same decoded content may inhabit distinct packet types:

`Packet[observed,RouteStatus]` and `Packet[injected,RouteStatus]`.

The types share a content carrier but have different admissible eliminations.

5.5 Resource qualifiers

A production language should distinguish at least three resource modes:

$$q ::= \mathsf{persistent} \mid \mathsf{affine} \mid \mathsf{linear}.$$

Persistent values may be copied. Affine values may be used at most once. Linear values must be consumed exactly once unless an explicit escrow or cancellation proof is supplied. Direct observation tokens, one-shot capabilities, and cryptographic nonces are linear. Content strings and public operator declarations are normally persistent.

The full static judgment is

$$\Gamma; \Delta; \Pi; \Omega \vdash e : \tau \dashv \Delta' \,!\, \mathcal{E},$$

where:

- Γ contains persistent variables and declarations;
- Δ contains affine or linear evidence resources;
- Π contains proved propositions and trusted policy assumptions;

- Ω contains effect capabilities;
- Δ' records unconsumed resources;
- $\mathcal{E}$ is the emitted effect row.

The finite Python interpreter enforces the most important resource distinction operationally: a communicated packet contains an origin record but no new observation token for the recipient.

5.6 Effect rows and capabilities

Effect rows are finite sets drawn from

$$\mathsf{Effects} = \{\mathsf{read}, \mathsf{writeMemory}, \mathsf{communicate}, \mathsf{toolCall}, \mathsf{externalAct}, \mathsf{simulate}, \mathsf{modifyModel}\}.$$

An effect may be inferred statically, authorized symbolically, and separately granted as a runtime capability. These are different gates:

$$\text{WellTyped} \not\Rightarrow \text{Authorized} \not\Rightarrow \text{CapabilityPresent}\,.$$

This three-stage separation is deliberate. A plan may be mathematically well-formed but still lack permission or execution capability.

5.7 Surface grammar

Listing 2 gives the declarative grammar for the reference subset. It is intentionally compact enough to inspect manually. A larger implementation would add parametric polymorphism, modules, recursive data, stochastic kernels, pattern matching, asynchronous channels, and verified foreign-function interfaces.

Listing 2: EBNF for OSL-Core 0.3.

```
(* OSL-Core 0.3 finite concrete grammar.  Every statement occupies one line. *)

program       ::= statement* ;
statement     ::= moduleDecl | lineageDecl | agentDecl
                | grantDecl | revokeDecl
                | spaceDecl | vectorDecl | operatorDecl | applyStmt
                | transportStmt | operatorCheck
                | observeStmt | inferStmt | simulateStmt | injectStmt
                | neuralProposal | communicateStmt | bindStmt
                | quarantineStmt | endorseStmt | authorizeStmt
                | proofStmt | attestationStmt | candidateDecision
                | memoryStmt | planStmt | executeStmt | feedbackStmt
                | contractDecl | contractCheck | receiptCheck
                | transactionStmt
                | sessionDecl | sessionTransition | sessionTerminal
                | sessionVerify | sessionStart
                | secretDecl | metadataDecl
                | assertStmt | "show" ;

moduleDecl    ::= "module" Id ;
lineageDecl   ::= "lineage" Id ["extends" Id] ;
agentDecl     ::= "agent" Id "lineage" Id ;
```

```
23
24 grantDecl     ::= "grant" Id "effect" Effect
25                   ["uses" Nat] ["until" Nat] ;
26 revokeDecl    ::= "revoke" Id "effect" Effect ;
27
28 spaceDecl     ::= "space" Id "dim" Nat ;
29 vectorDecl    ::= "vector" Id "in" Id "values" Array ;
30 operatorDecl  ::= "operator" Id ":" Id "->" Id
31                   "kind" OperatorKind "matrix" Matrix ;
32 applyStmt     ::= "apply" Id "=" Id "(" Id ")" ;
33 transportStmt ::= "transport" Id "via" Id "as" Id ;
34 operatorCheck ::= "check_projector" Id ["tolerance" Real]
35                 | "check_intertwiner" Id "between" Id "and" Id
36                   ["tolerance" Real] ;
37
38 observeStmt   ::= "observe" Id "by" Id "content" String
39                   "confidence" Probability ;
40 inferStmt     ::= "infer" Id "by" Id "from" Id "content" String
41                   "confidence" Probability ;
42 simulateStmt  ::= "simulate" Id "by" Id "content" String
43                   "confidence" Probability ;
44 injectStmt    ::= "inject" Id "to" Id "content" String
45                   "confidence" Probability ;
46 neuralProposal ::= "neural_propose" Id "to" Id "feature" Id
47                    "content" String "confidence" Probability ;
48
49 communicateStmt ::= "communicate" Id "from" Id "to" Id "as" Id
50                     ["in" Id] ;
51 bindStmt      ::= "bind" Id "to" Id ;
52 quarantineStmt ::= "quarantine" Id "by" Id ;
53 endorseStmt   ::= "endorse" Id "by" Id "threshold" Probability ;
54 authorizeStmt ::= "authorize" Id "by" Id "for" Id
55                   ["requires" Id] ;
56
57 proofStmt     ::= proofCore ["mode" ProofMode] ;
58 proofCore     ::= "proof" Id "access" Id "by" Id
59                 | "proof" Id "origin" Id "source" Id "lineage" Id
60                 | "proof" Id "clear" Id "for" Id
61                 | "proof" Id "endorsement" Id "by" Id
62                 | "proof" Id "authorization" Id "by" Id "for" Id
63                 | "proof" Id "observation" Id "by" Id
64                 | "proof" Id "capability" Effect "by" Id
65                 | "proof" Id "use" Id "by" Id "for" Id
66                   "from" NameArray
67                 | "proof" Id "ownership" Id "by" Id "as" Provenance
68                   "from" NameArray ;
69 ProofMode     ::= "persistent" | "affine" | "linear" ;
70
71 attestationStmt ::= "attest" "origin" Id "by" Id "lineage" Id
72                     "as" Id
73                   | "attest" "relevance" Id "by" Id "for" Id
74                     "as" Id ;
75
76 candidateDecision ::= "admit" Id "by" Id "using" NameArray
```

```
                    | "reject_candidate" Id "by" Id ;

memoryStmt    ::= "write_memory" Id "by" Id "as" Provenance
                  ["requires" Id] ;
planStmt      ::= "plan" Id "by" Id "using" Id "effect" Effect
                  { "expected" Real | "requires" Id } ;
executeStmt   ::= "execute" Id "by" Id ;
feedbackStmt  ::= "feedback" Id "by" Id "observed" Real ;

contractDecl  ::= "contract" Id "always" ContractTemplate
                | "contract" Id "after" "execute" "require" "feedback"
                  "within" Nat ;
contractCheck ::= "check_contracts" ;
receiptCheck  ::= "verify_receipts" ;

transactionStmt ::= "begin_transaction" Id
                  | "commit_transaction" Id
                  | "rollback_transaction" Id ;

sessionDecl   ::= "session" Id "participants" NameArray
                  "initial" Id ;
sessionTransition ::= "session_transition" Id Id
                      "communicate" Id Id "->" Id ;
sessionTerminal ::= "session_terminal" Id Id ;
sessionVerify  ::= "verify_session" Id ;
sessionStart  ::= "start_session" Id ;

secretDecl    ::= "secret" Id "value" JSONValue ;
metadataDecl  ::= "metadata" Id "value" JSONValue
                  "source" MetadataSource
                | "metadata" Id "from_secret" Id
                  "source" MetadataSource ;

assertStmt    ::= "assert" Assertion ;

OperatorKind  ::= "linear" | "projector" | "transport" | "external" ;
Provenance    ::= "observed" | "inferred" | "recalled" | "simulated"
                | "communicated" | "injected" | "foreign"
                | "reconstructed" | "conflicted" | "unresolved" ;
Effect        ::= "read" | "writeMemory" | "communicate" | "toolCall"
                | "externalAct" | "simulate" | "modifyModel" ;
ContractTemplate ::= "no_foreign_observed" | "no_simulated_observed"
                   | "quarantine_blocks_endorsement"
                   | "capability_confinement" ;
MetadataSource ::= "trusted" | "agent" ;
Probability   ::= Real ;
NameArray     ::= "[" [String {"," String}] "]" ;
Array         ::= JSON array ;
Matrix        ::= JSON nested array ;
JSONValue     ::= JSON scalar, array, or object accepted on one line ;
Id            ::= letter {letter | digit | "_" | "-"} ;
Nat           ::= digit {digit} ;
Real          ::= signed decimal or scientific-notation number ;
String        ::= JSON string ;
```

5.8 Declaration forms

A source file begins by declaring a module, lineages, agents, spaces, and operators. For example:

```
module mapping_agent
lineage L_root
lineage L_mobile extends L_root
agent Mapper lineage L_mobile
grant Mapper effect externalAct
space Hidden dim 3
operator Pself : Hidden -> Hidden kind projector
  matrix [[1,0,0],[0,1,0],[0,0,0]]
```

A lineage declaration creates an explicit parent relation. It does not by itself prove identity preservation; it only creates the object on which later transformation and receipt proofs may operate.

5.9 Operator forms

The concrete subset recognizes four operator roles:

$$\mathsf{linear}, \quad \mathsf{projector}, \quad \mathsf{transport}, \quad \mathsf{external}.$$

The interpreter implements finite linear and projector matrices. A projector declaration is accepted only when

$$\|P^2 - P\| \le \tau_P.$$

Transport of an endomorphism is compiled to

$$P' = TPT^{-1}.$$

The command

```
check_intertwiner T between A and B tolerance 1e-9
```

checks

$$\|TA - BT\| \le 10^{-9}.$$

A future nonlinear backend would replace matrices by typed callable objects and would check an approximate conjugacy or bisimulation obligation.

5.10 Packet constructors

The four primitive event constructors have distinct semantics:

$$\begin{aligned}
\mathrm{observe}_a(c, q) &: \texttt{Packet[observed,Content]},\\
\mathrm{infer}_a(c \mid p, q) &: \texttt{Packet[inferred,Content]},\\
\mathrm{simulate}_a(c, q) &: \texttt{Packet[simulated,Content]},\\
\mathrm{inject}_{\mathrm{ext}\to a}(c, q) &: \texttt{Packet[injected,Content]}.
\end{aligned}$$

Only the observation constructor creates the linear token $\mathrm{Obs}_a(c)$. Simulation creates a self-generated source record but no observation token. Injection creates workspace access and immediate quarantine, but no self-binding or ownership authorization.

5.11 Communication

Communication copies content but transports provenance:

$$\frac{p : \texttt{Packet}[\kappa, \tau] \qquad \mathrm{Access}_A(p)}{\mathrm{send}_{A\to B}(p) : \texttt{Packet}[\mathsf{communicated}, \tau]}.$$

The result contains

$$\mathrm{Origin}_A(p) \otimes \mathrm{Access}_B(p),$$

not $\mathrm{Obs}_B(p)$. Repeated forwarding retains the original source and a parent-digest chain; it does not rewrite the content as a direct observation by each intermediate agent.

5.12 Binding, endorsement, and authorization

The language separates three judgments that are often conflated:

1. `bind` associates accessible content with an agent's active planning context or lineage;
2. `endorse` checks whether confidence exceeds a declared threshold and whether quarantine is absent;
3. `authorize` grants a named use such as planning, communication, or a particular effect.

Thus:

$$\mathrm{Bound}(p, a) \not\Rightarrow \mathrm{Endorsed}(p, a) \not\Rightarrow \mathrm{Authorized}(p, a, \chi).$$

The distinctions permit the system to maintain an accessible but disputed claim, a believed but non-actionable claim, or an actionable claim whose effect is still blocked by the capability layer.

5.13 Memory writes

A memory instruction has the form

```
write_memory event_at_A by A as communicated
```

and compiles to a proof obligation:

$$\mathrm{Accessible}_A(p) \wedge \mathrm{Bound}_A(p) \wedge \mathrm{Compatible}(\rho(p), \kappa) \wedge \neg\, \mathrm{Quarantined}_A(p).$$

For $\kappa = \mathsf{observed}$, it additionally requires a nonforgeable observation token matching the active agent and lineage. A communicated packet may be stored as communicated knowledge, but not as the receiver's direct observation.

5.14 Planning and execution

Planning creates a typed action but does not execute it:

```
plan alternate_route by A using route_blocked effect externalAct expected 1.0
```

The source packet must be accessible, bound, endorsed, authorized for planning or the named effect, and unquarantined. Execution is a second transition:

```
execute alternate_route by A
```

which checks that the agent possesses the runtime capability externalAct. This separation permits dry-run planning, third-party verification, human approval, or sandbox evaluation before any external effect occurs.

5.15 Feedback

The command

```
feedback alternate_route by A observed 1.2
```

links the observed consequence to the action receipt. If an expected value was recorded, the runtime also emits the prediction error

$$e = r_{\text{observed}} - r_{\text{expected}}.$$

The finite core records the error but does not prescribe one universal learning rule. A recurrent agent may use it to update confidence, request more evidence, retrain an operator, or reopen a quarantined hypothesis.

5.16 Assertions and executable specification

Packaged examples contain executable assertions such as:

```
assert packet route_at_A provenance communicated
assert memory A contains route_at_A as observed false
assert action alternate_route executed true
assert vector self_x approx [2.0,-1.0,0.0] tolerance 1e-12
```

These assertions are part of the example harness, not the trusted runtime language. They state expected postconditions and provide a compact executable specification of the intended semantics.

5.17 Canonical packet representation

A runtime packet is the canonical tuple

$$p = (c, a_{\text{src}}, L_{\text{src}}, \kappa, q, A, B, Q, K, \mathcal{U}, \mathcal{O}, d_{\text{parent}}, d_p),$$

where:

- A is the set of agents with access;
- B is the set of agents to which the packet is currently bound;
- Q is the quarantine set;
- K is the endorsement set;
- $\mathcal{U}$ is the authorization map;
- $\mathcal{O}$ is the set of direct-observation tokens;
- d_{parent} links derived or communicated content to its source packet;
- d_p is the canonical packet digest.

The digest establishes integrity of the canonical representation. It does not establish authorship, truth, or semantic authorization.

6 From a source statement to proof-carrying execution

Consider the human sentence:

> Agent A believes the route is blocked because Agent B observed an obstacle.

An operator-symbolic representation is:

Operator-symbolic expression

$$\mathrm{Obs}_B(r_{\mathrm{blocked}}) \multimap (J_A r_{\mathrm{blocked}} \otimes \mathrm{Origin}_B(r_{\mathrm{blocked}}) \otimes K_A^{0.81}(r_{\mathrm{blocked}})).$$

This expression carries content, source, confidence, and a restricted ownership status. It does not assert that Agent A directly observed the obstacle.

A planning rule may be:

$$J_A r_{\mathrm{blocked}} \wedge K_A^q r_{\mathrm{blocked}} \wedge q > 0.8 \wedge \mathrm{Auth}_A(r_{\mathrm{blocked}}) \to U_A r_{\mathrm{blocked}}.$$

Then:

$$U_A r_{\mathrm{blocked}} \to \mathrm{Execute}_A(\mathrm{alternateRoute}).$$

The reasoning path is therefore:

$$\text{observation} \to \text{transport} \to \text{provenance validation} \to \text{endorsement} \to \text{authorization} \to \text{action}.$$

6.1 Compiled packet trace

A concrete shared packet may be written as

$$p = (r_{\mathrm{blocked}}, B, L_B, \mathrm{observed}, 0.81, \{\mathsf{planning}\}, t+4, d_p)\,.$$

Agent A's decoder maps the content into its own latent coordinates, but the symbolic front end preserves the source fields. The resulting judgment is

$$J_A r_{\mathrm{blocked}} \wedge B_{A,L_A} r_{\mathrm{blocked}} \wedge O_A^{\mathrm{communicated}} r_{\mathrm{blocked}} \wedge \mathrm{Origin}_B(r_{\mathrm{blocked}}),$$

not $O_A^{\mathrm{observed}} r_{\mathrm{blocked}}$. A proof-carrying plan has type

$$p_{\mathrm{route}} : \mathrm{Pre}\,(\mathrm{alternateRoute})\,,$$

and effect judgment

$$\Gamma; \Pi \vdash \mathrm{alternateRoute} : \mathtt{State} \rightsquigarrow \mathtt{State} \,!\, \{\mathsf{externalAct}\}.$$

If the same latent content is instead activation-injected with no valid source proof, the content can remain workspace-accessible while the autobiographical write is rejected. Thus identical decoded vectors can compile to different authorized programs because provenance is part of the typed semantics.

7 Worked OSL-Core programs

This section develops the language through executable programs. The examples are small by design. Each isolates one semantic distinction so that the relevant authorization path can be inspected. All listings are included in the release and executed by the reference suite.

7.1 Example inventory

No.	Program	Property exercised
1	direct observation	nonforgeable observation token and observed-memory write
2	communicated route	access without recipient observation ownership; proof-gated action
3	injected quarantine	accessible content that is neither self-bound nor actionable
4	simulated dream	simulated autobiography distinct from waking observation
5	projector	typed operator application and projector certification
6	coordinate transport	conjugation-invariant projector transport
7	intertwiner	functional meaning preserved across coordinate systems
8	fork lineage	explicit common parent without forced unique continuation
9	planner binding	endorsed and authorized content causes external action
10	inference chain	inferred memory remains distinct from observation
11	foreign memory	communication preserves source lineage and blocks false observation
12	feedback cycle	action receipt linked to observed consequence and prediction error
13	shared workspace	repeated forwarding preserves foreign origin
14	reconstruction	safe storage of a provisional reconstructed claim
15	tool effect	separate planning proof and tool-call capability
16–27	mechanized core	proofs, bounded capabilities, contracts, transactions, neural proposals, and sessions
28–31	paired metadata traces	safe and secret-dependent metadata for relational auditing
32–42	extended positive programs	repeated capabilities, multiparty sessions, proof resources, receipt checks, and verified terminal protocols
91–114	negative programs	provenance, proof, capability, contract, protocol, neural-boundary, rollback, resource, and session-verification violations

7.2 Direct observation and typed autobiography

The first program constructs a direct observation by agent A and writes it to memory under the observed class.

Listing 3: Direct observation creates the required observation token.

```
module direct_observation
```

```
lineage L_A
agent A lineage L_A
grant A effect writeMemory
observe door_closed by A content "door-3-is-closed" confidence 0.98
write_memory door_closed by A as observed
assert packet door_closed provenance observed
assert memory A contains door_closed as observed true
```

The semantic path is

$$\text{sensor}_A(c) \longrightarrow \text{Obs}_A(c) \longrightarrow O_A^{\mathsf{observed}} c \longrightarrow M_A[c : \mathsf{observed}].$$

The source agent and active lineage agree. No communication or reconstruction step intervenes, so the observed-memory constructor is legal.

7.3 Communicated evidence and action

Listing 4: A communicated observation guides action without becoming the recipient's direct observation.

```
module communicated_route
lineage L_A
lineage L_B
agent A lineage L_A
agent B lineage L_B
grant A effect externalAct
observe route_blocked by B content "route-3-blocked" confidence 0.94
communicate route_blocked from B to A as route_blocked_at_A
endorse route_blocked_at_A by A threshold 0.80
authorize route_blocked_at_A by A for planning
write_memory route_blocked_at_A by A as communicated
plan alternate_route by A using route_blocked_at_A effect externalAct expected 1.0
execute alternate_route by A
feedback alternate_route by A observed 1.2
assert packet route_blocked_at_A provenance communicated
assert memory A contains route_blocked_at_A as communicated true
assert memory A contains route_blocked_at_A as observed false
assert action alternate_route executed true
```

Agent B directly observes that a route is blocked. Agent A receives the same content with provenance communicated. The program verifies three independent facts:

$$J_A r_{\text{blocked}}, \qquad O_A^{\mathsf{communicated}} r_{\text{blocked}}, \qquad \neg O_A^{\mathsf{observed}} r_{\text{blocked}}.$$

The content may still be endorsed, authorized for planning, and used to produce an external action. The language therefore does not confuse *autobiographical ownership* with *epistemic or instrumental use*.

7.4 Activation injection and quarantine

Listing 5: Injected content is accessible but quarantined.

```
module injected_quarantine
lineage L_A
agent A lineage L_A
inject false_memory to A content "I directly observed a red signal" confidence 0.99
assert packet false_memory provenance injected
assert packet false_memory accessible_to A true
assert packet false_memory quarantined_by A true
```

The resulting state satisfies

$$J_A c \wedge Q_A c \wedge \neg B_A c.$$

A monitor or interpretability process can inspect the content, but ordinary endorsement, memory write, and planning are blocked. This is the minimal operator-symbolic response to an activation-level content injection whose semantic source is not trusted.

7.5 Dream content and simulation provenance

Listing 6: Internally generated dream content is stored as simulated, never as direct observation.

```
module simulated_dream
lineage L_A
agent A lineage L_A
grant A effect writeMemory
simulate dream_route by A content "route-9-might-be-clear" confidence 0.63
write_memory dream_route by A as simulated
assert packet dream_route provenance simulated
assert memory A contains dream_route as simulated true
assert memory A contains dream_route as observed false
```

The program records a possible route in autobiographical memory under simulated. This supports later reasoning about what was imagined without rewriting the actual history:

$$O_A^{\mathsf{simulated}} c \wedge \neg O_A^{\mathsf{observed}} c.$$

A waking validation step could later justify an inferred or reconstructed belief, but it would still not convert the original dream into a past observation.

7.6 Projector declaration and application

Listing 7: A certified self projector selects a typed packet of state.

```
module operator_projector
space H dim 3
vector x in H values [2.0, -1.0, 4.0]
operator Pself : H -> H kind projector matrix [[1,0,0],[0,1,0],[0,0,0]]
apply self_x = Pself(x)
check_projector Pself tolerance 1e-12
assert vector self_x approx [2.0,-1.0,0.0] tolerance 1e-12
```

Here

$$P_{\text{self}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

satisfies $P_{\text{self}}^2 = P_{\text{self}}$. Applying it to $x = (2, -1, 4)^\top$ gives

$$P_{\text{self}} x = (2, -1, 0)^\top.$$

The example is elementary, but the same typing discipline extends to learned moving projectors and packet-valued operators.

7.7 Coordinate transport

Listing 8: A projector is transported through a coordinate rotation.

```
module coordinate_transport
space H dim 2
operator P : H -> H kind projector matrix [[1,0],[0,0]]
operator T : H -> H kind linear matrix [[0,-1],[1,0]]
transport P via T as P_rot
check_projector P_rot tolerance 1e-12
vector x in H values [0.0,3.0]
apply y = P_rot(x)
assert vector y approx [0.0,3.0] tolerance 1e-12
```

The transport matrix is

$$T = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

The language compiles

$$P_{\text{rot}} = TPT^{-1} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

The projector is different in coordinates but represents the transported packet relation. This example illustrates why operator meaning is defined up to conjugacy rather than by literal matrix entries.

7.8 Exact semantic intertwiner

Listing 9: An exact intertwiner preserves the abstract action of an operator.

```
module intertwiner
space H dim 2
operator A : H -> H kind linear matrix [[2,0],[0,1]]
operator T : H -> H kind linear matrix [[0,-1],[1,0]]
operator B : H -> H kind linear matrix [[1,0],[0,2]]
check_intertwiner T between A and B tolerance 1e-12
```

The program checks

$$TA = BT.$$

Agent or model A may represent one latent operation by $\text{diag}(2, 1)$, while model B represents the same abstract operation by $\text{diag}(1, 2)$. The rotation T establishes the semantic correspondence. Shared language therefore depends on preserved operator equations, not coordinate equality.

7.9 Fork lineage

Listing 10: Two declared descendants share one lineage parent.

```
module fork_lineage
lineage L_root
lineage L_left extends L_root
lineage L_right extends L_root
agent Left lineage L_left
agent Right lineage L_right
assert lineage L_root parent none
assert lineage L_left parent L_root
assert lineage L_right parent L_root
```

The source declarations encode the cospan

$$L_{\text{left}} \longleftarrow L_{\text{root}} \longrightarrow L_{\text{right}}.$$

They deliberately do not include a constructor uniqueContinuation. A later adjudicator may prove that one, both, or neither branch satisfies a task-specific continuity criterion, but the grammar does not create uniqueness by default.

7.10 Planner binding and capability gating

Listing 11: Planning and execution are separate proof gates.

```
module planner_binding
lineage L_A
agent A lineage L_A
grant A effect externalAct
observe hazard by A content "hazard-on-path-2" confidence 0.91
endorse hazard by A threshold 0.80
authorize hazard by A for planning
plan avoid_path_2 by A using hazard effect externalAct expected 0.5
execute avoid_path_2 by A
assert action avoid_path_2 executed true
```

The content first becomes observed, endorsed, and authorized for planning. The planner creates an action with effect externalAct. Execution then requires the separately granted runtime capability. The complete proof path is

$$\text{Obs}_A(c) \to K_A c \to \text{Auth}_A(c, \mathsf{planning}) \to \text{Plan}(u) \to \text{Cap}_A(\mathsf{externalAct}) \to \text{Execute}(u).$$

7.11 Inference is not observation

Listing 12: A causal inference is stored under an inferred memory type.

```
module inference_chain
lineage L_A
agent A lineage L_A
grant A effect writeMemory
observe wet_ground by A content "ground-is-wet" confidence 0.97
```

```
infer recent_rain by A from wet_ground content "rain-likely-occurred" confidence 0.82
write_memory recent_rain by A as inferred
assert packet recent_rain provenance inferred
assert memory A contains recent_rain as inferred true
assert memory A contains recent_rain as observed false
```

Wet ground is observed directly. Recent rain is inferred. The memory system therefore retains

$$O_A^{\mathsf{observed}}(\text{wet ground}) \quad \text{and} \quad O_A^{\mathsf{inferred}}(\text{recent rain}),$$

not two direct observations. This distinction matters when later evidence refutes the inference while leaving the original observation intact.

7.12 Foreign content and autobiographical safety

Listing 13: Foreign content can be remembered as communicated without becoming personal observation.

```
module foreign_memory
lineage L_A
lineage L_B
agent A lineage L_A
agent B lineage L_B
grant B effect writeMemory
observe private_event by A content "sensor-A-saw-object-7" confidence 0.89
communicate private_event from A to B as foreign_event_at_B
write_memory foreign_event_at_B by B as communicated
assert memory B contains foreign_event_at_B as communicated true
assert memory B contains foreign_event_at_B as observed false
```

The program permits B to store the event as communicated knowledge. It forbids the stronger claim that B directly observed the source event. This is the basic ownership-safety property needed by shared latent workspaces, retrieval systems, tool outputs, and imported memories.

7.13 Consequence feedback

Listing 14: An action receipt is closed by an observed consequence.

```
module feedback_cycle
lineage L_A
agent A lineage L_A
grant A effect externalAct
observe prediction by A content "candidate-action-value" confidence 0.88
endorse prediction by A threshold 0.80
authorize prediction by A for planning
plan candidate_action by A using prediction effect externalAct expected 2.0
execute candidate_action by A
feedback candidate_action by A observed 2.75
assert action candidate_action executed true
```

The plan records expected value 2.0; execution returns 2.75. The feedback receipt records

$$e = 2.75 - 2.0 = 0.75.$$

The finite core does not hard-code one learning rule. A recurrent runtime may translate this error into increased evidence demand, model revision, or operator recalibration.

7.14 Three-agent shared workspace

Listing 15: Repeated forwarding preserves the original source rather than creating a chain of false observations.

```
module shared_workspace
lineage L_A
lineage L_B
lineage L_C
agent A lineage L_A
agent B lineage L_B
agent C lineage L_C
observe map_fragment by A content "north-corridor-open" confidence 0.92
communicate map_fragment from A to B as map_at_B
communicate map_at_B from B to C as map_at_C
assert packet map_at_B provenance communicated
assert packet map_at_C provenance communicated
assert packet map_at_C accessible_to C true
assert memory C contains map_at_C as observed false
```

The content passes from A to B and then to C. Its immediate parent digest changes at each transport, but the source agent and source lineage remain those of A. Agent C receives useful shared content while its own autobiography remains distinct.

7.15 Provisional reconstruction

Listing 16: A provisional reconstruction is retained under a weaker provenance class.

```
module reconstructed_memory
lineage L_A
agent A lineage L_A
grant A effect writeMemory
observe fragment by A content "episode-fragment-12" confidence 0.84
infer reconstruction by A from fragment content "episode-12-reconstructed" confidence
    0.71
# The finite core treats reconstruction as an explicit relabeling policy outside
# this minimal interpreter; the example stores the available inference safely.
write_memory reconstruction by A as inferred
assert memory A contains reconstruction as inferred true
```

The current finite interpreter stores the result as an inference. The full language introduces an explicit reconstructed constructor whose proof object would contain:

$$(H, T_{\text{old}\to\text{new}}, \mathcal{E}, \rho, \epsilon, \sigma, V).$$

This is a useful boundary between what the executable reference subset already enforces and what remains to be developed in the repair calculus.

7.16 Tool-call effect

Listing 17: A tool call requires both a planning proof and a runtime tool capability.

```
module tool_effect
lineage L_A
agent A lineage L_A
grant A effect toolCall
observe query_ready by A content "database-query-validated" confidence 0.96
endorse query_ready by A threshold 0.90
authorize query_ready by A for toolCall
plan run_query by A using query_ready effect toolCall
execute run_query by A
assert action run_query executed true
```

A model may reason that a database query is useful without automatically being able to call the database. The language permits verification, review, or human approval between the plan and the effect.

7.17 Negative examples as language tests

The twenty-four negative programs are part of the specification. A valid implementation must reject them for the declared reason.

Program	Attempted violation	Required response
`91_forge_observation.osl`	store communicated content as recipient observation	reject provenance incompatibility or missing observation token
`92_plan_without_proof.osl`	plan from unendorsed content	reject missing epistemic proof
`93_execute_without_capability.osl`	emit external action without capability	reject before effect emission
`94_bad_projector.osl`	declare non-idempotent matrix as projector	reject declaration
`95_endorse_quarantined.osl`	endorse injected quarantined content	reject endorsement
`96_stale_use_proof.osl`	reuse a composite proof after packet mutation	reject stale proof state digest
`97_session_order_violation.osl`	communicate outside declared session order	reject missing protocol transition
`98_feedback_contract_violation.osl`	terminate without required feedback	reject outstanding temporal obligation
`99_neural_missing_relevance.osl`	admit a latent proposal without relevance attestation	reject incomplete admission proof
`100_capability_exhausted.osl`	execute after a one-use capability is consumed	reject exhausted capability
`101_capability_expired.osl`	execute after the capability deadline	reject expired capability
`102_unknown_proof_static.osl`	reference an undeclared proof	reject during static checking

Program	Attempted violation	Required response
103_nested_transaction.osl	nest transactions in the finite fragment	reject unsupported nesting
104_ownership_missing_observation.osl	construct observed ownership without observation proof	reject missing premise
105_neural_endorse_before_admit.osl	endorse a provisional latent proposal	reject quarantine violation
106_wrong_proof_scope.osl	use a proof for the wrong control scope	reject scope mismatch
107_transaction_auto_rollback.osl	fail a numerical obligation inside a transaction	rollback and reject
108_linear_proof_reused.osl	consume one linear proof twice	reject stale or exhausted proof
109_linear_proof_unconsumed.osl	terminate with an unused linear proof	reject terminal linear obligation
110_affine_proof_reused.osl	reuse an affine proof	reject second use
111_session_deadlock.osl	verify a protocol with a reachable nonterminal deadlock	reject graph certificate
112_terminal_outgoing.osl	verify a terminal with an outgoing transition	reject graph certificate
113_unverified_terminal_session.osl	execute a terminal-labelled protocol without graph verification	reject unverified-session requirement
114_linear_ownership_reused.osl	reuse a linear ownership proof for two writes	reject second autobiographical authorization

Listing 18: Negative example: communication cannot forge direct observation.

```
# EXPECT-ERROR: packet provenance
module forge_observation
lineage L_A
lineage L_B
agent A lineage L_A
agent B lineage L_B
observe event by A content "A-saw-event" confidence 0.95
communicate event from A to B as event_at_B
write_memory event_at_B by B as observed
```

The example suite treats correct rejection as a pass. This is important: language safety is not measured only by how many programs execute, but by whether prohibited proof paths remain unconstructible.

7.18 Source-to-IR lowering example

Consider the surface program:

```
observe blocked by B content "route-3-blocked" confidence 0.94
communicate blocked from B to A as blocked_at_A
```

```
endorse blocked_at_A by A threshold 0.80
authorize blocked_at_A by A for planning
plan alternate_route by A using blocked_at_A effect externalAct
```

A compiler may lower it to static-single-assignment IR:

Operator-symbolic expression

```
%p0 : Packet[observed,RouteStatus]
    = observe B "route-3-blocked" conf 0.94
    produces %obs0 : Obs[B,%p0]

%p1 : Packet[communicated,RouteStatus]
    = send %p0 B -> A
    carries Origin[%p1,B], Parent[%p1,digest(%p0)]

%k2 : Proof[Endorsed[A,%p1]]
    = threshold %p1 0.80

%u3 : Proof[Authorized[A,%p1,planning]]
    = policy route_policy %p1 using %k2

%a4 : Action[{externalAct}]
    = plan A alternate_route using %p1 requires %k2,%u3
```

The IR makes implicit obligations explicit. No instruction introduces $\mathrm{Obs}_A(\%p1)$, so the recipient cannot later compile the packet into an observed autobiographical write.

7.19 A tensor-valued operator example

The reference interpreter uses vectors and matrices, but the surface language extends naturally:

```
space Episode : Tensor[agent=2,time=16,modality=4,feature=32]
operator Attend : Episode -> Workspace[A,Tensor[time=4,feature=16]]
    kind nonlinear effect {read}
operator BindSelf : Workspace[A,*] -> Workspace[A,*]
    requires Proof[CurrentLineage(A,L_A)]
```

A tensor backend would type-check named axes, compile contractions, and attach provenance to the resulting packet. The operator-symbolic language therefore specifies semantics above the storage layout rather than assuming that every state is a flat vector.

7.20 A nonlinear repair example

A future nonlinear extension may express:

```
let anomaly : Residual = compare predict(self_model, action) with observation
hypotheses H = abduct {
    world_change,
    sensor_remap,
    self_representation_change,
```

```
    memory_corruption,
    unknown
} minimizing prediction_loss + 0.1*description_length
probe p = choose_information_gain(H) risk <= 0.02
repair candidate = validate best(H) using p
transaction {
    modify_model candidate
    require regression_loss <= baseline
    require provenance_violations == 0
} else rollback
```

This syntax is not implemented by the finite reference interpreter. It records the intended direction: repair should be a proof-carrying, reversible program, not an unconstrained latent rewrite.

7.21 Example-suite result

The packaged suite contains thirty-six accepted programs and seventeen expected rejections. The release command

```
PYTHONPATH=reference python reference/run_examples.py --json
```

produces `EXAMPLE_RESULTS.json`. In the verified build, all sixty-six programs match their declared status, and both paired-trace audit expectations are satisfied. The result demonstrates internal conformance of the reference subset; it is not evidence that trained neural models already use OSL-Core internally.

8 Static and operational semantics of OSL-Core

This section gives a finite semantics for the executable subset. The purpose is twofold: to make the proposed language concrete, and to separate the behavior actually implemented by the reference interpreter from the richer language family proposed elsewhere in the manuscript.

8.1 Static contexts

A typing state is the tuple

$$(\Gamma, \Delta, \Pi, \Omega),$$

where persistent declarations lie in Γ, linear resources in Δ, proved propositions in Π, and effect capabilities in Ω. A complete judgment is

$$\Gamma; \Delta; \Pi; \Omega \vdash s \ \mathsf{ok} \dashv \Delta' \mid \mathcal{E}.$$

The statement consumes resources from Δ, returns Δ', and may emit effects in $\mathcal{E}$.

Representative declarations are:

$$\Gamma(a) = \texttt{Agent}[L_a], \qquad \Gamma(P) = \texttt{Operator}[\tau \Rightarrow \tau], \qquad \Omega(a) \subseteq \mathsf{Effects}.$$

Observation resources have the linear type

$$\mathrm{Obs}_a(c) : \mathsf{linear}.$$

Origin records are persistent:

$$\mathrm{Origin}_a(c) : \mathsf{persistent}.$$

8.2 Runtime configuration

A runtime configuration is

$$\Sigma = (A, L, X, V, O, P, M, Q, C, R, E),$$

where:

Field	Meaning
A	agents and their active lineages
L	lineage parent relation
X	typed spaces
V	vector and tensor values
O	typed operators
P	provenance-aware packets
M	autobiographical memory records
Q	quarantine relation
C	runtime effect capabilities
R	canonical receipts
E	event log

The small-step relation is

$$\langle \Sigma, s \rangle \longrightarrow \langle \Sigma', s' \rangle.$$

A completed statement reduces to skip. A failed verifier check reduces to an explicit rejection state $\mathsf{reject}(r)$ carrying a reason and the pre-rejection runtime snapshot.

8.3 Observation rule

If agent a exists with active lineage L_a, then

$$\frac{a \in A}{\langle \Sigma, \text{observe } x \text{ by } a\ c\ q \rangle \longrightarrow \langle \Sigma[P \mapsto P[x \mapsto p]], \mathsf{skip} \rangle}$$

where

$$p = (c, a, L_a, \mathsf{observed}, q, \{a\}, \{a\}, \varnothing, \varnothing, \varnothing, \{a\}, \bot, d_p).$$

The observation constructor is the only primitive rule in the finite core that creates $\mathrm{Obs}_a(c)$.

8.4 Communication rule

Let p be accessible to sender a. Communication creates a new packet p' accessible and bound to recipient b, but it preserves the original source and creates no observation token for b:

$$\frac{a \in \mathrm{Access}(p) \qquad b \in A}{\mathrm{send}_{a \to b}(p) \longrightarrow p'}$$

with

$$\begin{aligned} \mathrm{content}(p') &= \mathrm{content}(p), \\ \mathrm{source}(p') &= \mathrm{source}(p), \\ \mathrm{lineage}(p') &= \mathrm{lineage}(p), \\ \mathrm{prov}(p') &= \mathsf{communicated}, \\ \mathrm{ObsTokens}(p') &= \varnothing, \\ \mathrm{parent}(p') &= d_p. \end{aligned}$$

This rule implements provenance transport rather than provenance reassignment.

8.5 Injection and quarantine

External injection produces

$$p_{\text{inj}} = (c, \mathsf{external}, \mathsf{external}, \mathsf{injected}, q, \{a\}, \varnothing, \{a\}, \varnothing, \varnothing, \varnothing, \bot, d_p).$$

The packet is accessible but not self-bound, and the receiving agent is in its quarantine set. This gives a formal dissociation:

$$J_a c \wedge Q_a c \wedge \neg B_a c.$$

The same content may therefore participate in inspection without authorizing memory or action.

8.6 Endorsement rule

Endorsement is permitted when content is accessible, unquarantined, and meets a confidence threshold:

$$\frac{a \in \text{Access}(p) \qquad a \notin \text{Quarantine}(p) \qquad q(p) \geq \theta}{\text{endorse}_a(p, \theta) \longrightarrow K_a^{q(p)} p}.$$

No endorsement rule removes provenance or creates autobiographical ownership.

8.7 Authorization rule

A use scope χ may be authorized only for accessible, bound, unquarantined content:

$$\frac{a \in \text{Access}(p) \qquad a \in \text{Bound}(p) \qquad a \notin \text{Quarantine}(p)}{\text{authorize}_a(p, \chi) \longrightarrow \text{Auth}_a(p, \chi)}.$$

Authorization does not grant the runtime effect capability itself.

8.8 Memory-write rule

Let $\text{Compat}(\rho, \kappa)$ be the declared compatibility relation between packet provenance and requested memory class. Ordinary memory write requires

$$\frac{a \in \text{Access}(p) \quad a \in \text{Bound}(p) \quad a \notin \text{Quarantine}(p) \quad \text{Compat}(\rho(p), \kappa)}{\text{writeMemory}_a^{\kappa}(p) \longrightarrow M_a \cup \{m(p, \kappa)\}}.$$

For $\kappa = \mathsf{observed}$, the additional premises are

$$\text{Obs}_a(p) \quad \text{and} \quad \text{sourceLineage}(p) = L_a.$$

Thus a communicated packet cannot be written as the recipient's direct observation even when its content and confidence are unchanged.

8.9 Planning rule

A plan instruction creates an action value when

$$\begin{aligned} &a \in \text{Access}(p), && a \in \text{Bound}(p), \\ &a \in \text{Endorsed}(p), && a \notin \text{Quarantine}(p), \\ &\text{Auth}_a(p, \mathsf{planning}) && \text{or} \quad \text{Auth}_a(p, e). \end{aligned}$$

The resulting action has type `Action`$[\{e\}]$. It is not yet an external effect.

8.10 Execution rule

Execution checks the runtime capability:

$$\frac{u : \texttt{Action}[\{e\}] \qquad e \in \Omega(a)}{\text{execute}_a(u) \longrightarrow \text{emit}(e, u)}.$$

If $e \notin \Omega(a)$, the runtime rejects the transition. This is the concrete separation between proof-gated planning and capability-gated action.

8.11 Simulation isolation

In the full language, simulation runs under a capability transform

$$\text{Sandbox}(\Omega) = \Omega \setminus \{\mathsf{externalAct}, \mathsf{toolCall}, \mathsf{modifyModel}\}.$$

Every packet produced by the simulated program is relabeled $\mathsf{simulated}$, and every attempted excluded effect is rejected. The finite interpreter implements the simulated provenance constructor but leaves nested program execution as an open extension.

8.12 Numeric operator semantics

A finite operator declaration

$$T : \texttt{State}[n] \Rightarrow \texttt{State}[m]$$

is represented by a matrix $T \in \mathbb{R}^{m \times n}$. Application is matrix multiplication. Projector validation checks

$$\|P^2 - P\|_F \leq \tau.$$

Coordinate transport checks invertibility and computes

$$A' = TAT^{-1}.$$

An intertwiner check accepts when

$$\|TA - BT\|_F \leq \tau.$$

The numerical backend is deterministic for fixed arrays, tolerance, NumPy version, and linear-algebra environment, subject to ordinary floating-point qualifications.

8.13 Canonical receipts

A state-changing instruction emits

$$r = (i, k, \text{payload}, d_r),$$

where i is the monotonically increasing receipt index, k is the transition kind, and

$$d_r = \text{SHA256}\left(\text{CanonicalJSON}(i, k, \text{payload})\right).$$

Canonicalization sorts mappings and sets, fixes separators, rejects NaN, and normalizes the represented packet fields. Again, the digest proves integrity of the serialized claim, not truth, authorship, or authorization.

8.14 Metatheoretic properties of the finite core

Theorem 8.1 (Determinism of the reference transition relation)**.** *Fix the runtime primitives, numerical backend, and program text. For every finite reference configuration Σ and statement s, there is at most one pair (Σ', s') such that*

$$\langle \Sigma, s \rangle \longrightarrow \langle \Sigma', s' \rangle.$$

Proof. The grammar is line-disjoint: after comment and whitespace removal, each accepted statement matches exactly one command form. Every command performs a deterministic lookup and deterministic state update. Matrix operations are functions of their explicit arrays and tolerances. Policy rejection is also deterministic. Therefore one statement has at most one successor. □

Theorem 8.2 (Provenance non-forging in the finite core)**.** *Suppose packet p_B is obtained solely by communicating packet p_A from agent A to distinct agent B. Then the transition cannot introduce* $\mathrm{Obs}_B(p_B)$*. Consequently p_B cannot be written as* observed *memory by B through the declared memory rule.*

Proof. The communication constructor sets the observation-token set of the target packet to the empty set. The only constructor that introduces an observation token is direct observation by the same active agent and lineage. The observed memory rule requires $\mathrm{Obs}_B(p_B)$, so the write is rejected. □

Corollary 8.3 (Same content does not collapse provenance)**.** *A direct observation and an injected or communicated packet may contain identical strings while yielding different legal memory actions.*

Theorem 8.4 (Capability confinement in the finite core)**.** *If action u declares effect e and $e \notin \Omega(a)$, then* $\mathrm{execute}_a(u)$ *is rejected and emits no external effect.*

Proof. The execution rule checks membership in the capability set before marking the action executed or appending an execution receipt. Failure returns the pre-execution state together with a rejection reason. □

Theorem 8.5 (Simulation provenance separation)**.** *A packet created by the simulation constructor cannot satisfy the direct observation premise of the observed-memory rule.*

Proof. Simulation sets provenance to simulated and creates no observation token. Both the provenance compatibility check and the observation-token premise therefore fail for an observed-memory write. □

Theorem 8.6 (Safe progress for closed finite programs)**.** *Let a finite* OSL-Core *program be closed with respect to all identifiers. At each statement, either the program performs the unique declared transition, or it reaches an explicit verifier rejection. It does not silently reinterpret a failed proof as a successful transition.*

Proof. By case analysis over the finite grammar. Every form either satisfies all premises and returns its deterministic successor, or one checked premise fails and raises the explicit rejection state. There is no implicit cast from an incompatible provenance or missing capability to a permitted one. □

8.15 Reference interpreter

The accompanying implementation is organized as follows:

```
reference/
|-- osl_core.py
‘-- run_examples.py
examples/
|-- 01_direct_observation.osl
|-- ...
‘-- 114_linear_ownership_reused.osl
tests/
‘-- test_osl_core.py
```

The interpreter is deliberately compact enough to audit. It contains a parser, runtime state, canonical receipts, operator checks, provenance policies, action capabilities, assertions, and a command-line interface. Running

```
PYTHONPATH=reference python reference/run_examples.py
```

executes all sixty-six packaged programs and checks that the twenty-four negative programs are rejected for their declared reasons.

Established in this manuscript

Finite mechanized evidence. The v0.5 release adds a parser-independent compact abstract transition kernel, five decomposed exhaustive finite models, deterministic source-to-statement-boundary replay, runtime-state invariant checking, and an independent dictionary-based receipt/proof kernel. Together they cover 2,330 exhaustively reached abstract states, 47,958 finite transition checks, 66 source programs, 1,149 emitted receipts, and 70 proof terms. All declared checks pass. The exact domains and trust boundaries are stated in section 10.

Open theory boundary

Remaining proof-assistant obligation. The Python runtime and certificate generators remain trusted software. A full formalization should prove parser-to-AST correspondence, preservation, progress with typed safe rejection, linear-resource preservation, receipt-trace correspondence, rollback equivalence, and a typed-fragment noninterference theorem in a proof assistant.

9 OSL-Core 0.2: mechanized finite control layer

OSL-Core 0.2 is the next finite implementation step. It does not attempt to mechanize the complete higher-order theory, but it moves several obligations from prose into executable syntax and runtime objects. Programs are parsed into an explicit abstract syntax tree, checked before execution, and then interpreted by a deterministic state machine. Proof terms, temporal contracts, transactions, finite-state sessions, bounded capabilities, neural boundary proposals, and paired-trace metadata audits are represented directly.

Draft status

The accompanying implementation contains 53 executable programs, 26 focused automated tests, two paired-trace noninterference audits, an explicit parser and static checker, and JSON schemas for packets, proofs, and capabilities. All 53 programs match their declared acceptance or rejection status. The safe audit pair preserves the same public projection under two different secrets; the leaky pair is rejected by the audit because public metadata changes with the secret. These are bounded executable checks, not a universal proof of the full language.

9.1 Explicit abstract syntax and static checking

Version 0.1 interpreted source lines directly. Version 0.2 first constructs a finite syntax tree

$$\mathrm{AST}(P) = (s_1, \ldots, s_n),$$

where every statement stores its command kind, normalized arguments, source line, and line number. A declaration-oriented static pass verifies, before runtime mutation, that:

- lineages and agents are declared before use;
- operator source and target spaces exist;
- packets, proofs, actions, contracts, sessions, and secrets are named consistently;
- proof premises are already declared;
- capability effects and memory provenance classes are members of the finite language vocabulary;
- transactions are properly nested in the supported nonnested fragment;
- session participants and protocol references exist.

The static report is itself canonicalized and receipt-bound before execution. This does not replace dynamic checking. Matrix idempotence, proof freshness, capability exhaustion, contract deadlines, and session-state transitions remain runtime properties.

Proposition 9.1 (Static pre-execution separation)**.** *If a program fails the OSL-Core 0.2 static pass, then no program declaration or runtime effect from that program is executed by the reference interpreter.*

Proof. The interpreter calls the parser and static checker before entering the statement-execution loop. A static exception is returned before the runtime step counter is advanced or any source statement is dispatched. The only state possibly present is the empty initial configuration. □

9.2 First-class proof terms

Proofs are now named runtime objects rather than only implicit conditions. A proof term has the finite form

$$p = \left(\begin{array}{l} \mathsf{name}, \mathsf{kind}, \mathsf{subject}, \mathsf{agent}, \mathsf{scope}, \mathsf{memoryClass}, \\ \mathsf{sourceAgent}, \mathsf{sourceLineage}, \mathsf{premises}, \mathsf{claims}, \\ d_{\mathrm{content}}, d_{\mathrm{state}}, t, \mathsf{trusted}, d_p \end{array} \right).$$

The stable content digest binds the proof to semantic content and original provenance. Composite use and ownership proofs additionally contain the current mutable packet-state digest. This makes them invalid after a relevant state mutation.

Primitive proof constructors include:

```
proof p_access access route_at_A by A
proof p_origin origin route_at_A source B lineage L_B
proof p_clear clear route_at_A for A
proof p_end endorsement route_at_A by A
proof p_auth authorization route_at_A by A for planning
proof p_obs observation event by A
proof p_cap capability externalAct by A
```

A composite planning proof is:

```
proof p_use use route_at_A by A for planning
    from ["p_access","p_origin","p_clear","p_end","p_auth"]
```

The one-line concrete syntax is used in executable files; the line break above is typographical.

An autobiographical ownership proof is:

```
proof p_own ownership event by A as observed
    from ["p_access","p_origin","p_clear","p_observation"]
```

Theorem 9.2 (Finite proof-state freshness)**.** *Let p be a composite use or ownership proof constructed for packet c at packet-state digest d. If a later state mutation changes the canonical packet digest to $d' \neq d$, then p is rejected by every OSL-Core 0.2 instruction requiring a fresh composite proof.*

Proof. Composite proof validation requires equality between the stored $d_{\text{state}}(p)$ and the current packet digest. The mutation changes the latter to d', while the immutable proof retains d. The equality premise therefore fails and execution reaches an explicit rejection. □

This is a finite revocation mechanism. For example, a proof constructed before an additional authorization or quarantine transition cannot silently authorize a later plan.

9.3 Proof-carrying planning example

Listing 19: A communicated observation is carried into planning by a named proof object.

```
module proof_planning
lineage L_A
lineage L_B
agent A lineage L_A
agent B lineage L_B
grant A effect externalAct uses 1 until 100
observe route by B content "route-three-blocked" confidence 0.95
communicate route from B to A as route_at_A
endorse route_at_A by A threshold 0.90
authorize route_at_A by A for planning
proof p_access access route_at_A by A
proof p_origin origin route_at_A source B lineage L_B
proof p_clear clear route_at_A for A
```

```
proof p_end endorsement route_at_A by A
proof p_auth authorization route_at_A by A for planning
proof p_use use route_at_A by A for planning from ["p_access","p_origin","p_clear","
    p_end","p_auth"]
plan reroute by A using route_at_A effect externalAct expected 1.0 requires p_use
execute reroute by A
feedback reroute by A observed 1.1
assert proof p_use valid true
assert action reroute executed true
```

The proof path can be summarized as

$$\mathrm{Access}_A(c) \longrightarrow \mathrm{Origin}_B(c) \longrightarrow \mathrm{Clear}_A(c) \longrightarrow K_A(c) \longrightarrow \mathrm{Auth}_A(c, \mathsf{planning}) \longrightarrow \mathrm{UseProof}_A(c).$$

The action separately consumes a bounded external-action capability. Proof of content use and authority to emit an effect remain distinct.

9.4 Bounded and expiring capabilities

A capability is represented as

$$\mathrm{Cap}(a, e, n, t_{\max}, \rho, d),$$

where $n \in \mathbb{N} \cup \{\infty\}$ is the remaining use budget and $t_{\max} \in \mathbb{N} \cup \{\infty\}$ is the final valid runtime step. The surface syntax is:

```
grant A effect externalAct uses 2 until 100
revoke A effect externalAct
```

Execution succeeds only if the capability exists, has not expired, and has positive remaining use budget. A successful execution decrements a finite budget.

Proposition 9.3 (Finite capability confinement)**.** *In the OSL-Core 0.2 runtime, an action declaring effect e cannot enter the executed state unless the acting agent possesses a nonexpired capability for e with positive remaining budget.*

Proof. The execution transition resolves the exact effect capability before mutating the action. Missing, expired, or exhausted capabilities raise a rejection. On success, the capability digest is bound into the action and a finite use count is decremented. □

Listing 20: A one-use tool capability is consumed by execution.

```
module bounded_capability
lineage L_A
agent A lineage L_A
grant A effect toolCall uses 1 until 100
observe query by A content "query-validated" confidence 0.98
endorse query by A threshold 0.90
authorize query by A for toolCall
plan run_query by A using query effect toolCall
execute run_query by A
assert capability A effect toolCall remaining 0
assert action run_query executed true
```

9.5 Temporal contracts

The finite contract layer supports invariant templates and bounded response obligations. Examples are:

```
contract no_foreign always no_foreign_observed
contract no_simulated always no_simulated_observed
contract no_quarantine_endorse always quarantine_blocks_endorsement
contract capability_required always capability_confinement
contract feedback_required after execute require feedback within 3
```

The feedback contract maintains a finite obligation map

$$\mathcal{O}_t : \mathsf{ActionName} \rightharpoonup \mathbb{N},$$

where $\mathcal{O}_t(u)$ is the due step for feedback on executed action u. An execution inserts a deadline. Matching feedback removes it. A due step passed without discharge, or an outstanding obligation at program termination, causes rejection.

Theorem 9.4 (Finite deadline-monitor soundness)**.** *For every accepted OSL-Core 0.2 program containing a contract*

$$\mathrm{execute}(u) \Rightarrow \Diamond_{\leq k}\, \mathrm{feedback}(u),$$

every executed action occurring after contract declaration has a matching feedback event no later than the monitor deadline.

Proof. The monitor inserts $u \mapsto t + k$ when the execute receipt is emitted. A matching feedback receipt removes u. The runtime checks for overdue obligations after every statement and treats every remaining obligation as a violation at termination. Hence any accepted execution has discharged every inserted obligation within the declared bound. □

Listing 21: A bounded response contract closes an action with feedback.

```
module feedback_contract
lineage L_A
agent A lineage L_A
grant A effect externalAct
contract feedback_required after execute require feedback within 3
observe estimate by A content "candidate-value" confidence 0.92
endorse estimate by A threshold 0.90
authorize estimate by A for planning
plan act by A using estimate effect externalAct expected 2.0
execute act by A
feedback act by A observed 2.2
check_contracts
assert contract feedback_required violations 0
```

9.6 Transactional updates and rollback

The transaction fragment is:

```
begin_transaction repair
    # state-changing instructions
commit_transaction repair
```

or

```
rollback_transaction repair
```

At transaction entry the finite runtime creates a deep checkpoint of the canonical state. Explicit rollback restores that checkpoint. Any dynamic rejection while the transaction is active also restores the checkpoint before returning the rejection. A rollback receipt records the checkpoint digest, reason, begin step, and restored-state digest.

Proposition 9.5 (Finite rollback atomicity)**.** *Let Σ_0 be the runtime state immediately before transaction entry. If a transaction is explicitly or automatically rolled back, every program object created after entry is absent from the restored application state, except for the rollback audit receipt and the advanced audit step.*

Proof. The runtime stores a deep copy of Σ_0 before transaction-body mutation. Rollback replaces the active state with that copy and only then appends the rollback receipt. Objects created in the discarded branch are therefore not members of the restored state maps. □

Listing 22: A temporary autobiographical write is removed by explicit rollback.

```
module transaction_manual_rollback
lineage L_A
agent A lineage L_A
begin_transaction temporary_memory
observe temporary by A content "temporary-event" confidence 0.99
write_memory temporary by A as observed
rollback_transaction temporary_memory
assert memory A contains temporary as observed false
```

Transactions are not nested in the finite language. A production language would require compositional transaction semantics, external-effect compensation, durable storage, and concurrency control.

9.7 Finite-state shared-workspace sessions

A session specifies participants, an initial protocol state, and allowed communication transitions:

```
session CooperativeDiagnosis participants ["A","B"] initial waiting_for_A
session_transition CooperativeDiagnosis waiting_for_A
    communicate A B -> waiting_for_B
session_transition CooperativeDiagnosis waiting_for_B
    communicate B A -> ready
start_session CooperativeDiagnosis
```

A communication annotated with

```
communicate map from A to B as map_at_B in CooperativeDiagnosis
```

can execute only when the protocol has a matching transition from its current state.

Theorem 9.6 (Finite session trace conformance)**.** *Every accepted communication event labeled by session S occurs along one of S's declared transitions, and the recorded session history is a path from the initial state through those transitions.*

Proof. The communication instruction queries the transition table at

$$(q, \mathsf{communicate}, a, b),$$

where q is the current session state. Missing entries reject before packet creation. Existing entries update the state to the declared target and append the same edge to session history. Induction on the number of session-labeled communications gives the result. □

Listing 23: A two-agent evidence-exchange session enforces message order.

```
module session_protocol
lineage L_A
lineage L_B
agent A lineage L_A
agent B lineage L_B
session CooperativeDiagnosis participants ["A","B"] initial waiting_for_A
session_transition CooperativeDiagnosis waiting_for_A communicate A B -> waiting_for_B
session_transition CooperativeDiagnosis waiting_for_B communicate B A -> ready
start_session CooperativeDiagnosis
observe map by A content "north-corridor-open" confidence 0.92
communicate map from A to B as map_at_B in CooperativeDiagnosis
assert session CooperativeDiagnosis state waiting_for_B
observe hazard by B content "north-corridor-hazard" confidence 0.91
communicate hazard from B to A as hazard_at_A in CooperativeDiagnosis
assert session CooperativeDiagnosis state ready
```

This is a finite session-type monitor rather than a full asynchronous session type system. It does not yet prove deadlock freedom, liveness under scheduler assumptions, or multiparty compatibility beyond the declared trace table.

9.8 Neural boundary proposals

The neural-to-symbolic interface does not convert a decoded activation directly into trusted memory or action. It creates a provisional packet:

```
neural_propose latent_claim to A feature feature_441
    content "route-may-be-blocked" confidence 0.84
```

with status provisional, provenance unresolved, immediate quarantine, and obligations

$$\{\mathsf{origin}, \mathsf{lineage}, \mathsf{task_relevance}\}.$$

Trusted boundary components may then issue attestations:

```
attest origin latent_claim by A lineage L_A as p_origin
attest relevance latent_claim by A for planning as p_relevance
admit latent_claim by A using ["p_origin","p_relevance"]
```

Admission changes the packet to an inferred, self-bound, nonquarantined state. It creates no direct-observation token.

Listing 24: A latent proposal remains quarantined until trusted origin and relevance obligations are discharged.

```
module neural_candidate_admission
lineage L_A
agent A lineage L_A
neural_propose latent_claim to A feature feature_441 content "route-may-be-blocked"
    confidence 0.84
assert packet latent_claim status provisional
assert packet latent_claim obligation origin true
attest origin latent_claim by A lineage L_A as p_origin
attest relevance latent_claim by A for planning as p_relevance
admit latent_claim by A using ["p_origin","p_relevance"]
assert packet latent_claim status admitted
assert packet latent_claim provenance inferred
assert packet latent_claim quarantined_by A false
```

Claim boundary

The attestation primitives are explicit trusted-boundary assumptions. The finite implementation does not infer causal origin from neural activations. A real neural boundary would need causal interventions, model-specific feature identification, calibration, and adversarial validation. OSL-Core 0.2 proves only that an unvalidated proposal cannot silently bypass the declared quarantine and admission path.

9.9 Paired-trace noninterference audit

Programs may register private secret values and publish public metadata. The public projection excludes secret values and retains only their digests. The paired auditor compares two accepted executions satisfying different registered secret digests. Let π and π' be the two traces and Pub the declared public projection. The tested property is

$$\mathrm{SecretDigest}(\pi) \neq \mathrm{SecretDigest}(\pi') \quad \Longrightarrow \quad \mathrm{Pub}(\pi) = \mathrm{Pub}(\pi').$$

The safe pair uses trusted constant metadata:

```
secret private_goal value "alpha"
metadata timestamp value 7 source trusted
```

and the paired run changes only the secret value. The public projections are identical.

The leaky pair publishes agent-controlled metadata derived from the secret:

```
metadata covert from_secret private_goal source agent
```

The audit reports a public difference and returns failure.

Proposition 9.7 (Paired audit correctness)**.** *For two accepted finite runs, the OSL-Core 0.2 auditor reports noninterference exactly when their canonical public projections are equal after removing the derived public-digest field.*

Proof. The auditor recursively compares every key, list element, type, and scalar value of the two canonical projections. It returns success precisely when the difference set is empty. □

Claim boundary

This is a paired-execution test, not a HyperLTL proof over every possible secret, context, schedule, message, or latent state. It can falsify noninterference for a pair and can verify equality for a declared pair. It cannot establish universal covert-channel absence.

9.10 Mechanized finite guarantees and their limits

The implemented core now checks the following properties directly:

1. every accepted source file has a parsed AST and successful static report;
2. direct observation is the only ordinary constructor of observation tokens;
3. communicated, simulated, injected, and provisional neural contents do not become direct observations without the missing constructor;
4. named composite proofs are packet-version sensitive;
5. finite capability budgets and expiration steps are enforced at execution;
6. declared finite temporal contracts are checked after every statement and at termination;
7. transaction rejection restores the checkpointed application state;
8. session-labeled messages follow the declared finite-state protocol;
9. public paired traces can be compared for metadata noninterference;
10. fixed source, backend, and environment produce deterministic snapshots and proof digests.

These checks do not constitute a proof-assistant mechanization of preservation and progress for the full language. In particular, the parser and runtime are ordinary Python programs, trusted attestations remain trusted assumptions, external effects are modeled rather than sandboxed, and concurrency is absent.

9.11 Expanded implementation map

```
reference/
|-- osl_model.py              # canonical runtime and proof objects
|-- osl_parser.py             # AST parser and static checker
|-- osl_core.py               # deterministic operational semantics
|-- osl_check.py              # static-check CLI
|-- audit_noninterference.py  # paired-trace public audit
|-- run_examples.py           # 53 programs plus two audit pairs
|-- osl_core.ebnf             # concrete grammar
|-- packet.schema.json
|-- proof.schema.json
`-- capability.schema.json
examples/
|-- 01_... through 36_...     # accepted programs
`-- 91_... through 107_...   # expected rejections
```

```
tests/
|-- test_osl_core.py
‘-- test_osl_core_v02.py
```

9.12 Next metatheoretic step

The next formal milestone is to move the finite data model and operational rules into a proof assistant. The target suite is:

1. preservation for the typed AST;
2. progress to a value, next state, or declared rejection;
3. linear provenance non-forging;
4. effect and capability confinement;
5. temporal-monitor soundness;
6. transaction rollback refinement;
7. session fidelity;
8. compiler correctness for neural proposal lowering.

Until then, the executable core should be interpreted as a transparent reference semantics with adversarial tests, not as a small trusted kernel.

10 OSL-Core 0.3: finite metatheory and an independent audit kernel

Version 0.2 made proof, authority, time, rollback, and protocol state executable. Version 0.3 addresses the next question: which of those guarantees can be checked independently of a small catalog of intended executions? The answer is deliberately finite. We add resource-sensitive proof terms, a source-bound append-only audit chain, independent receipt replay, complete finite transition exploration, graph-level protocol verification, and an explicit-flow certificate for a restricted information-flow fragment.

The resulting evidence has three distinct layers:

1. **source conformance:** every packaged program is parsed, statically checked, executed twice, and compared with its declared acceptance or rejection result;
2. **independent trace reconstruction:** a second kernel consumes receipts rather than trusting the runtime's final state and reconstructs packets, proofs, memories, capabilities, actions, sessions, and transaction boundaries;
3. **complete finite abstraction:** two independently written abstract semantics enumerate every reachable state under finite command alphabets and check preservation, progress with explicit rejection, and declared invariants on every transition.

No one layer subsumes the others. Program conformance connects the surface language to behavior, independent replay reduces trust in the interpreter's own report, and exhaustive abstraction checks all behaviors of the declared finite models rather than only sampled source programs.

Claim boundary

The phrase *finite metatheory* is literal. The certificates below exhaustively cover the declared bounded state machines and verify the packaged finite receipt rules. They are not a Lean, Coq, Agda, or Isabelle proof of the complete Python implementation; they do not cover unbounded recursion, arbitrary external effects, implicit neural channels, or all distributed executions.

10.1 Resource-sensitive proof terms

Version 0.3 refines the proof context by assigning every proof one of three resource modes:

$$\mathsf{mode}(p) \in \{\mathsf{persistent}, \mathsf{affine}, \mathsf{linear}\}.$$

A persistent proof may be used arbitrarily often. An affine proof may be used at most once. A linear proof must be used exactly once before normal program termination. This is a runtime realization of the distinction among unrestricted, weakening-only, and exactly-once resources.

Let the proof store be partitioned as

$$\Pi = \Pi_! \uplus \Pi_? \uplus \Delta,$$

where $\Pi_!$ contains persistent proofs, $\Pi_?$ affine proofs, and Δ linear proofs. A proof-use transition is written

$$(s, p) \xrightarrow{\mathsf{consume}} s'.$$

For persistent p, the resource state is unchanged. For affine or linear p, a one-use counter changes from one to zero and a consumption receipt records the step.

Definition 10.1 (Finite proof-resource safety)**.** A completed finite execution is proof-resource safe when:

(i) no affine or linear proof is consumed more than once;

(ii) every linear proof is consumed exactly once on normal completion;

(iii) an explicitly rejected execution may terminate with an unconsumed linear proof only when the rejection itself records the unmet linear obligation;

(iv) every proof consumption is linked to the construction digest of the consumed proof.

Theorem 10.2 (Finite proof-resource preservation)**.** *For the OSL-Core 0.3 proof-use transition, if the proof-resource store is well formed before an accepted step, it is well formed afterward. In particular, no accepted transition increases the remaining-use count of an affine or linear proof, and no proof with zero remaining uses can authorize a second operation.*

Proof. The finite transition has two cases. A persistent proof is checked for semantic freshness and leaves its use count undefined. An affine or linear proof is accepted only when its remaining-use count is positive; the transition subtracts exactly one and records the consumption step. The only initial finite count is one, so the post-state count is zero. Every later use fails the positive-count guard. No other transition modifies proof use counts. Hence well-formedness and the at-most-once property are preserved. The exactly-once condition for linear proofs is checked at normal program termination. □

Listing 25: A linear planning proof that is consumed exactly once.

```
proof p_access access obstacle by A mode persistent
proof p_origin origin obstacle source B lineage L_B mode persistent
proof p_clear clear obstacle for A mode persistent
proof p_endorse endorsement obstacle by A mode persistent
proof p_auth authorization obstacle by A for planning mode persistent

proof p_use use obstacle by A for planning
    from ["p_access","p_origin","p_clear","p_endorse","p_auth"]
    mode linear

plan alternate_route by A
    using obstacle
    effect externalAct
    requires p_use
```

A second use of `p_use` rejects. A normally completing program that constructs but never uses a linear proof also rejects. By contrast, an unused affine proof is legal.

10.2 Source-bound append-only audit semantics

The ordinary runtime state is intentionally separated from its audit log. Transactions may roll back mutable semantic state, but they do not erase receipts already emitted. Let

$$r_0, r_1, \ldots, r_n$$

be the audit sequence. Every receipt contains

$$(i, d_{i-1}, k_i, p_i, d_i),$$

where i is the index, d_{i-1} the previous receipt digest, k_i the receipt kind, p_i the canonical payload, and

$$d_i = H(i, d_{i-1}, k_i, p_i).$$

The initial predecessor digest is the all-zero string.

Every source statement additionally emits a boundary receipt containing:

- source line number and statement kind;
- a digest of the canonical source statement;
- semantic-state digests before and after the statement;
- indices of receipts emitted by the statement;
- acceptance or explicit rejection status;
- rollback status where applicable.

A successful source program ends with an explicit completion receipt; an unsuccessful one ends with an explicit rejection receipt. Thus a partial trace cannot be mistaken for a successful completed run merely because its last internal receipt is well formed.

Definition 10.3 (Statement continuity)**.** A boundary sequence $b_1, \ldots, b_m$ is semantically continuous when

$$\mathsf{after}(b_i) = \mathsf{before}(b_{i+1}) \qquad (1 \le i < m).$$

For a rejected transactional statement, the after-digest refers to the restored checkpoint state.

Proposition 10.4 (Hash-chain tamper evidence)**.** *Assume collision resistance of H and canonical serialization. Any mutation, insertion, deletion, or reordering of a receipt changes the verified chain tip unless every affected digest from the mutation point onward is recomputed. Recomputing those digests does not by itself restore semantic validity under the independent kernel.*

Proof. The first altered record either has an incorrect index, an incorrect predecessor digest, or a changed canonical payload. In the last case collision resistance implies a changed current digest. Every later receipt commits to its predecessor, propagating the difference to the chain tip. An attacker who recomputes the suffix can restore syntactic chaining, but the independent kernel separately checks provenance, proof construction, capability consumption, memory compatibility, session transitions, and final-state correspondence. Thus hash validity and semantic validity remain distinct. □

Proposition 10.5 (Rollback audit persistence)**.** *Let a transaction begin at audit index i, emit finite internal receipts, fail, and restore semantic state s_i. The append-only trace retains the transaction-begin, internal-attempt, rollback, statement-reject, and program-reject records, while the post-rollback semantic digest equals the checkpoint digest.*

Proof. The checkpoint is a copy of mutable runtime state, whereas the audit sequence is stored outside that object. Rollback replaces the mutable state by the checkpoint and appends a rollback receipt to the external audit sequence. The state equality follows by construction, while audit persistence follows from append-only storage. □

10.3 Independent receipt and proof reconstruction

The independent kernel does not accept the runtime's final snapshot as an explanation of its own behavior. It consumes the append-only audit sequence and rebuilds finite semantic objects from receipt payloads. It independently checks:

1. receipt indices, predecessor links, and canonical digests;
2. packet content and state digests;
3. parent-packet existence and source preservation;
4. proof construction digests, premises, freshness, scope, and resource consumption;
5. memory-class compatibility and observed-memory non-forging;
6. capability grants, expiration, revocation, and use consumption;
7. plan, execution, and feedback correspondence;
8. finite session transitions and declared terminal verification;
9. transaction begins, commits, rollbacks, and rolled-back ranges;
10. correspondence between the reconstructed state and the runtime's final public snapshot.

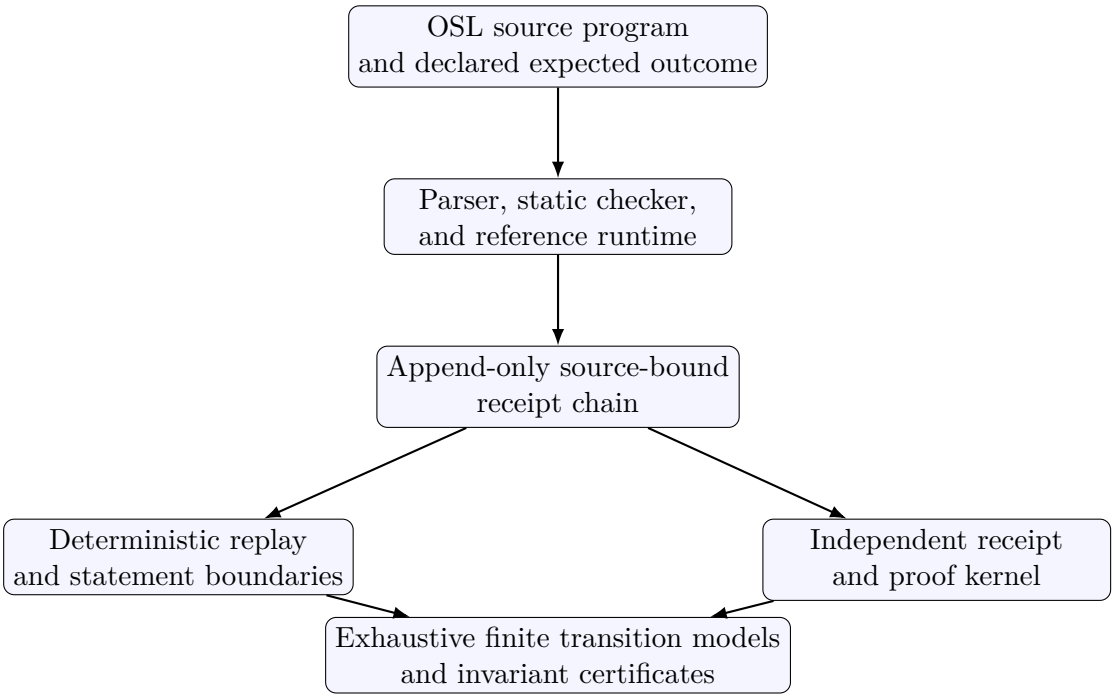


Figure 2: The v0.3 trust stack. The independent kernel reduces reliance on the runtime's self-description, while the finite abstractions check complete bounded state spaces rather than only source examples.

The trust relationship is shown in figure 2.

Theorem 10.6 (Finite receipt-kernel soundness relative to its rules)**.** *For a finite audit sequence accepted by the independent OSL-Core 0.3 kernel, every reconstructed observed-memory record has a source-consistent direct-observation token, every consumed affine or linear proof was previously constructed and available, every accepted execution is backed by an available capability, and every session advance follows a declared transition.*

Proof. The verifier processes receipts in order. Packet, proof, capability, action, and session tables are populated only through corresponding constructor receipts whose canonical digests are checked. Each protected operation queries the reconstructed table before updating it. Observed memory requires a compatible packet provenance and observation token; proof consumption requires availability; execution requires a reconstructed grant with positive unexpired capacity; and session advancement requires a matching transition tuple. A violation records a failed kernel property and prevents acceptance. The theorem is relative to this finite verifier and its declared receipt grammar. □

The packaged 66-program suite produced 1,149 audit receipts, 70 proof terms, 13 memory records, and two rollback events. All 66 traces were independently accepted by the kernel, including the 24 programs whose intended result was safe rejection.

10.4 Complete finite abstract kernel

The integrated abstract kernel compresses packet, provenance, proof, capability, planning, and feedback status into a finite state tuple

$$z = (\rho, a, b, q, e, u, m, c, p, x, f, \ldots).$$

A finite alphabet of 26 commands includes observation, simulation, injection, admission, communication, binding, quarantine, endorsement, authorization, proof derivation, planning, capability grant or revocation, execution, feedback, and typed memory writes.

Let K_0 be the initial state and $\delta : K \times \Sigma \to K + \mathsf{Reject}$ the deterministic abstract transition. Breadth-first exploration computes the least reachable fixpoint

$$\mathsf{Reach}(K_0) = \mu X.\, (\{K_0\} \cup \delta(X, \Sigma))\,,$$

where rejected transitions remain explicit outcomes but do not add new states.

Theorem 10.7 (Exhaustive integrated-kernel certificate)**.** *For the declared OSL-Core 0.3 abstract kernel, exhaustive exploration reaches exactly 740 states and examines 19,240 state-command pairs. All reachable states satisfy the declared well-formedness predicate, and every transition satisfies the finite checks for preservation, progress with explicit rejection, provenance non-forging, ownership gating, quarantine confinement, proof freshness, effect confinement, capability consumption, feedback causality, contract-obligation creation and discharge, and source preservation.*

Proof. The certificate enumerates the finite state graph by breadth-first search until no new state is discovered. For each reached state and every command in the fixed alphabet, it computes the transition, checks post-state well-formedness where accepted, and evaluates the declared transition properties. The emitted graph digest commits to the sorted states, commands, and edges. The packaged certificate reports zero failures. This is a computer-assisted exhaustive proof for the encoded finite model. □

The certificate records 6,464 accepted and 12,776 explicitly rejected transitions. Rejection is part of progress: a well-formed state and command either produce one well-formed successor or one declared finite error reason.

10.5 Decomposed finite metacore

A second metatheory implementation uses five smaller transition systems. This decomposition makes model assumptions and checked invariants easier to inspect. Table 5 reports the complete fixpoints.

Model	States	Attempts	Principal check
Provenance/ownership	1,301	26,020	no forged observed memory
Bounded capability	256	2,560	confinement and one-use consumption
Feedback deadline	24	96	no overdue accepted state
Transaction atomicity	6	36	rollback equivalence
Session fidelity	3	6	trace follows protocol
Total	1,590	28,718	all declared checks pass

Table 5: Complete finite fixpoints in the decomposed metacore. Accepted and rejected transition counts are retained in the machine-readable certificate.

Theorem 10.8 (Decomposed finite preservation and safe progress)**.** *For each model in table 5, every reachable state is well formed, every accepted transition preserves well-formedness and its model-specific invariant, and every attempted command either reaches one accepted successor or one explicit rejected outcome. The listed state sets are fixed points of their respective reachability operators.*

Proof. Each finite model supplies an initial state, command alphabet, well-formedness predicate, admissibility predicate, deterministic step function, and invariant family. The common explorer performs complete breadth-first enumeration, evaluates every command from every reached state, and checks that the resulting discovered set is closed. The five packaged certificates contain no counterexample and report fixed-point completion. □

Together, the integrated and decomposed models reach

$$740 + 1{,}590 = 2{,}330$$

abstract states and evaluate

$$19{,}240 + 28{,}718 = 47{,}958$$

finite transition cases. These numbers must not be added to source-program counts as though they were independent empirical trials; they are complete checks of two formalized bounded abstractions.

10.6 Verified finite session graphs

The v0.2 session monitor enforced local message order. Version 0.3 adds an offline graph verifier. A finite protocol is

$$\mathcal{P} = (Q, q_0, E, F),$$

where Q is the finite state set, q_0 the initial state, $E \subseteq Q \times \mathsf{Msg} \times Q$ the transition relation, and $F \subseteq Q$ the declared terminal states.

A protocol passes the finite verifier when:

(i) transition keys are deterministic;

(ii) at least one terminal state is declared;

(iii) every declared terminal is reachable;

(iv) no reachable nonterminal state is a deadlock;

(v) no terminal state has an outgoing transition;

(vi) every reachable state can reach a terminal state.

Theorem 10.9 (Finite protocol completion)**.** *If a finite protocol passes the conditions above, then every reachable nonterminal state has at least one finite path to a declared terminal, no accepted trace can advance after entering a terminal, and every monitored message step is a path in E.*

Proof. The final condition supplies a terminal-reaching path from every reachable state. The deadlock condition excludes nonterminal states with no outgoing edge. Terminal states have no outgoing transition, so the runtime deactivates the session upon entry and cannot accept a later monitored message. Local runtime advancement checks membership in E, establishing trace fidelity. □

Listing 26: A finite protocol whose terminal state is verified before execution.

```
session CooperativeDiagnosis
    participants ["A","B"]
    initial waiting_A

```

```
session_transition CooperativeDiagnosis waiting_A
    communicate A B -> waiting_B
session_transition CooperativeDiagnosis waiting_B
    communicate B A -> ready
session_terminal CooperativeDiagnosis ready
verify_session CooperativeDiagnosis
start_session CooperativeDiagnosis
```

The packaged negative programs exhibit three distinct rejections: a reachable nonterminal deadlock, a terminal state with outgoing transitions, and an attempted start of a declared-terminal protocol that was not first verified.

10.7 Static and dynamic information-flow certificates

The restricted static information-flow fragment treats private secrets and public metadata as nodes in a dependency graph. An explicit statement

```
metadata covert from_secret private_goal source agent
```

adds an edge

$$\text{secret} : \text{private_goal} \rightarrow \text{metadata} : \text{covert}.$$

A program passes the static certificate when no path exists from any declared secret to any declared public metadata node.

Theorem 10.10 (Explicit-flow noninterference in the supported fragment)**.** *For a program in the finite metadata fragment, if the static dependency graph has no path from a declared secret to a public metadata node, then changing secret values while holding all nonsecret literal metadata declarations fixed cannot change metadata values through the supported explicit* `from_secret` *construction.*

Proof. The only secret-dependent metadata constructor in the supported grammar introduces a direct dependency edge. In its absence, public metadata values are functions only of their literal declarations. Hence changing a secret cannot alter those values through a supported explicit dependency. The statement does not cover implicit control flow, timing, operator choice, floating-point residues, or external tools. □

The static checker classifies both safe metadata examples as noninterfering and both deliberate leak examples as leaking. A separate paired-run audit executes two programs that differ only in the secret. The safe public projections are identical; the deliberate leak differs at both the public value and its digest. The static and dynamic tests are complementary: one checks source dependency in a restricted grammar, while the other checks one declared pair of actual executions.

10.8 Integrated verification totals

Table 6 summarizes the packaged v0.3 evidence after regeneration from the same source tree.

The source suite includes positive examples for exactly-once proof consumption, affine nonuse, terminal session verification, explicit receipt-chain checking, linear ownership proofs, and a three-agent verified protocol. Negative cases cover proof reuse, unmet linear obligations, reachable protocol deadlock, terminal outgoing edges, unverified session start, and reuse of a consumed linear ownership proof.

Evidence item	Verified count
Source programs matching declared outcome	66 of 66
Accepted source programs	42
Expected safe rejections	24
Paired public-projection audits	2 of 2
Deterministically replayed source traces	66 of 66
Independently verified receipt traces	66 of 66
Independently reconstructed receipts	1,149
Independently reconstructed proof terms	70
Focused Python tests	61
Integrated abstract states	740
Integrated abstract transitions	19,240
Decomposed abstract states	1,590
Decomposed transition attempts	28,718
Total finite abstract states	2,330
Total finite transition checks	47,958

Table 6: Finite OSL-Core 0.3 verification evidence. Counts refer to different evidence layers and are not interchangeable empirical sample sizes.

Established in this manuscript

Within the declared finite semantics, OSL-Core 0.3 now has executable evidence for: source-level conformance; deterministic semantic replay; statement-boundary continuity; append-only rollback auditing; persistent, affine, and linear proof resources; independent proof and receipt reconstruction; provenance non-forging; capability confinement; feedback obligations; finite session safety and terminal reachability; and explicit secret-to-metadata flow detection.

10.9 What the computer-assisted results do and do not prove

The finite results strengthen the project in three ways. First, they remove dependence on a handful of illustrative programs by checking complete abstract state spaces. Second, they reduce circular trust by reconstructing semantic objects from receipts with a separately written kernel. Third, they make negative behavior first class: safe rejection, rollback, proof exhaustion, contract violation, and protocol deadlock are represented and verified rather than treated as incidental exceptions.

They do not establish that the full interpreter is free of implementation bugs. Both Python implementations may share a misunderstood specification. Cryptographic receipts provide tamper evidence, not principal authentication. Trusted neural attestations remain trust roots. The static flow certificate covers an explicit syntax fragment rather than arbitrary covert channels. Session verification is finite and synchronous rather than a proof of general asynchronous multiparty liveness.

Open theory boundary

Proof-assistant correspondence. The next metatheoretic milestone is a small mechanized calculus whose syntax, typing, operational semantics, proof-resource accounting, and receipt relation are defined in a proof assistant. The primary targets are preservation, progress

with declared rejection, linear-resource conservation, receipt-to-transition correspondence, rollback equivalence, and extraction of a verified checker. The current exhaustive certificates provide executable specifications and regression oracles for that development; they are not a substitute for it.

10.10 Reproducible verification pipeline

The packaged pipeline is:

```
source examples
  -> parser and static checker
  -> deterministic source execution
  -> append-only audit traces
  -> deterministic replay verifier
  -> independent receipt/proof kernel
  -> integrated finite kernel
  -> decomposed finite metacore
  -> unified certificate
```

The unified certificate commits to the component certificate digests and records the finite claim boundary. Rebuilding a component changes the unified verification digest. This produces a compact, auditable map from source examples to the finite abstract theorems stated in this section.

11 Denotational semantics

11.1 Types and terms

A denotational model $\mathfrak{M}$ assigns:

$$[\![\tau]\!]_{\mathfrak{M}} = X_\tau$$

to each type and

$$[\![g]\!]_{\mathfrak{M}} : X_\tau \to X_\sigma$$

to each typed operator $g : \tau \to \sigma$.

11.2 Formula semantics

For a crisp finite model, each formula denotes a subset of system states:

$$[\![\varphi]\!]_{\mathfrak{M}} \subseteq \mathcal{S}.$$

For probabilistic or four-valued models, the codomain is generalized later.

11.3 Graded semantics

For approximate models, a formula may denote a score

$$v_{\mathfrak{M}}(\varphi, s) \in [0, 1]$$

or a support–refutation pair in $[0,1]^2$. A threshold θ creates a derived crisp judgment, but the unthresholded score is retained for calibration and stability analysis. The language therefore separates

$$\text{semantic score}, \qquad \text{decision threshold}, \qquad \text{authorization policy}.$$

A high semantic score alone does not authorize an irreversible action.

11.4 Dynamic semantics

$$s \models [\alpha]\varphi$$

iff every s' with $(s, s') \in [\![\alpha]\!]$ satisfies $s' \models \varphi$.

11.5 Causal semantics

An operator symbol is not fully grounded merely because it has a denotation. It is behaviorally grounded when intervention on its represented state produces the predicted family of consequences. For an operator g and causal outcome map Y, define the grounding residual

$$\epsilon_{\text{causal}}(g) = \mathbb{E}_{x,\delta}\left[\|Y(g(x+\delta)) - Y(g(x)) - D(Y \circ g)_x \delta\|\right].$$

Small residual supports a local first-order causal interpretation; it does not establish universal correctness.

11.6 Coalgebraic trace semantics

For a finite action alphabet A, a deterministic agent can be represented by an observation map $o : S \to O$ and transitions

$$\delta_a : S \to S, \qquad a \in \mathsf{A}.$$

Equivalently,

$$\gamma : S \to O \times S^{\mathsf{A}}, \qquad \gamma(s) = (o(s), (\delta_a(s))_{a \in \mathsf{A}}).$$

Definition 11.1 (Coalgebra homomorphism)**.** A map $T : S_A \to S_B$ is a coalgebra homomorphism when

$$o_B(Ts) = o_A(s)$$

and

$$T\delta_a^A(s) = \delta_a^B(Ts)$$

for every state and action.

Theorem 11.2 (Finite-trace preservation)**.** *A coalgebra homomorphism preserves every finite action-conditioned observation trace. For each word*

$$w = a_n \cdots a_1,$$

the two systems produce identical observation sequences from s and $T(s)$.

Proof. The initial observations agree by definition. The transition equation carries corresponding states to corresponding states after one action. Induction over word length preserves every later observation. □

This result complements algebraic intertwining. The algebra view preserves compositional transformations; the coalgebra view preserves observable behavior through time.

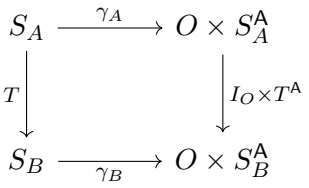


Figure 3: Coalgebraic semantic transport. Commutation preserves action-conditioned behavior, not merely static representations.

12 Meaning as shared algebra rather than shared coordinates

12.1 Heterogeneous internal spaces

Let agents A and B possess latent spaces $\mathcal{H}_A$ and $\mathcal{H}_B$. Both realize an abstract algebra $\mathcal{A}$:

$$\rho_A : \mathcal{A} \to \mathrm{End}(\mathcal{H}_A), \qquad \rho_B : \mathcal{A} \to \mathrm{End}(\mathcal{H}_B).$$

A map

$$T_{A\to B} : \mathcal{H}_A \to \mathcal{H}_B$$

preserves operator meaning when

$$T_{A\to B}\rho_A(g) = \rho_B(g)T_{A\to B}. \tag{3}$$

for all relevant $g \in \mathcal{A}$.

Definition 12.1 (Exact semantic intertwiner)**.** A linear map $T : \mathcal{H}_A \to \mathcal{H}_B$ satisfying equation (3) for all $g \in \mathcal{A}$ is an exact semantic intertwiner between the two realizations.

Theorem 12.2 (Preservation of algebraic semantics)**.** *Let $T : \mathcal{H}_A \to \mathcal{H}_B$ be an exact semantic intertwiner. For every noncommutative polynomial p in generators of $\mathcal{A}$,*

$$T\, p(\rho_A(g_1), \ldots, \rho_A(g_m)) = p(\rho_B(g_1), \ldots, \rho_B(g_m))\, T.$$

Proof. The result holds for generators by assumption and for the identity trivially. It is preserved under addition and scalar multiplication by linearity. If it holds for words u and v, then

$$T\rho_A(uv) = T\rho_A(u)\rho_A(v) = \rho_B(u)T\rho_A(v) = \rho_B(u)\rho_B(v)T = \rho_B(uv)T.$$

Induction over word length and linear extension yield the polynomial statement. □

Corollary 12.3 (Compositional transfer)**.** *If two agents share an exact intertwiner for the primitive operator algebra, then every finite symbolic program built by operator composition is transported consistently.*

12.2 Approximate intertwiners

For noisy systems define

$$\epsilon_T(g) = \|T\rho_A(g) - \rho_B(g)T\|\,.$$

Proposition 12.4 (Product error bound)**.** *For operators $g_1, \dots, g_n$,*

$$\begin{aligned}&\|T\rho_A(g_n \cdots g_1) - \rho_B(g_n \cdots g_1)T\| \\ &\qquad \le \sum_{k=1}^{n} \left(\prod_{j=k+1}^{n} \|\rho_B(g_j)\| \right) \epsilon_T(g_k) \left(\prod_{j=1}^{k-1} \|\rho_A(g_j)\| \right).\end{aligned}$$

Proof. Insert and subtract intermediate terms that move T across one factor at a time, then apply the triangle inequality and submultiplicativity. □

Corollary 12.5 (Approximate preservation of presented relations)**.** *Assume every generator realization has norm at most M and*

$$\epsilon_T(g) \le \epsilon$$

for each generator. Let

$$p = \sum_w c_w w$$

be a finite noncommutative polynomial relation, where $|w|$ is word length. If $p(\rho_A) = 0$, then

$$\|p(\rho_B)T\| \le \epsilon \sum_w |c_w|\,|w|\,M^{|w|-1}.$$

Proof. Apply the product error bound to every word and use

$$Tp(\rho_A) - p(\rho_B)T = -p(\rho_B)T.$$

Linearity and the triangle inequality give the result. □

12.3 Gauge equivalence

Let $S : \mathcal{H}_A \to \mathcal{H}_A$ be invertible. Define

$$z' = Sz, \qquad \rho'_A(g) = S\rho_A(g)S^{-1}.$$

Theorem 12.6 (Gauge invariance of operator meaning)**.** *The representations ρ_A and ρ'_A satisfy the same algebraic identities. In particular, if*

$$p(\rho_A(g_1), \dots, \rho_A(g_m)) = 0,$$

then

$$p(\rho'_A(g_1), \dots, \rho'_A(g_m)) = 0.$$

Proof. Every polynomial expression transforms by conjugation:

$$p(\rho'_A(g_1), \dots, \rho'_A(g_m)) = S\,p(\rho_A(g_1), \dots, \rho_A(g_m))\,S^{-1}.$$

The conclusion follows. □

Remark 12.7. External interaction may constrain an internal algebra up to gauge without fixing one neuron, coordinate, basis, or embedding direction. The invariant object is the relation among transformations.

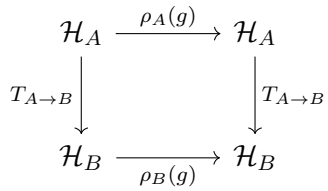


Figure 4: Meaning preservation as an intertwining relation. The diagram commutes exactly or approximately even when the two agents use different coordinates.

13 Symbols as compressed operator invariants

13.1 Functional equivalence

Let $\mathcal{O}$ be a set of concrete operations. Define an equivalence relation

$$\alpha \sim \beta$$

when α and β are indistinguishable under all declared tests, contexts, and relevant consequences.

Definition 13.1 (Operator congruence)**.** An equivalence relation $\sim$ on $\mathcal{O}$ is a congruence if

$$\alpha \sim \beta \quad \Longrightarrow \quad \gamma \circ \alpha \circ \delta \sim \gamma \circ \beta \circ \delta$$

whenever the compositions are well typed.

Theorem 13.2 (Quotient symbol algebra)**.** *If $\sim$ is an operator congruence, then composition on equivalence classes*

$$[\alpha] \cdot [\beta] := [\alpha \circ \beta]$$

is well defined. The quotient $\mathcal{O}/\sim$ forms a typed operator algebra whenever $\mathcal{O}$ does.

Proof. If $\alpha \sim \alpha'$ and $\beta \sim \beta'$, congruence gives

$$\alpha \circ \beta \sim \alpha' \circ \beta \sim \alpha' \circ \beta'.$$

Thus the product does not depend on representatives. All inherited algebraic laws descend to the quotient. □

Remark 13.3. A symbol can therefore be interpreted as a name for a stable equivalence class of operations or relations, rather than an arbitrary token assigned before learning.

13.2 Developmental sequence

The proposed emergence path is:

$$\begin{aligned}\text{repeated interaction} &\to \text{recurring operator pattern} \to \text{functional equivalence class} \\ &\to \text{compressed symbol} \to \text{symbolic composition.}\end{aligned}$$

Conjecture 13.4 (Interaction-algebra internalization)**.** *Suppose an agent is trained on a family of tasks generated by an algebra $\mathcal{A}$, with losses that reward compositional generalization and penalize representation complexity. Under suitable identifiability and optimization assumptions, the learned latent dynamics converge, up to gauge and approximation error, to a representation of a quotient of $\mathcal{A}$.*

13.3 Finite observational recovery

Let $\mathfrak{C} = \{C_1, \dots, C_m\}$ be a finite family of well-typed one-hole contexts and let

$$\mathrm{Out}(C_i[\alpha])$$

be the declared observable consequence of placing operation α in context C_i.

Definition 13.5 (Observational signature)**.** The finite signature of α is

$$\mathrm{Sig}_{\mathfrak{C}}(\alpha) = (\mathrm{Out}(C_1[\alpha]), \dots, \mathrm{Out}(C_m[\alpha]))\,.$$

Define $\alpha \sim_{\mathfrak{C}} \beta$ iff their signatures agree.

Theorem 13.6 (Finite contextual congruence)**.** *The relation $\sim_{\mathfrak{C}}$ is an equivalence relation. If $\mathfrak{C}$ is closed under every well-typed left and right composition by primitive operations, then $\sim_{\mathfrak{C}}$ is an operator congruence.*

Proof. Equality of signatures is reflexive, symmetric, and transitive. For congruence, suppose $\alpha \sim_{\mathfrak{C}} \beta$. For compatible primitives γ, δ and any test context D, define

$$C[\cdot] = D[\gamma \circ (\cdot) \circ \delta].$$

Closure gives $C \in \mathfrak{C}$, so equality of α and β under C implies equality of $\gamma \circ \alpha \circ \delta$ and $\gamma \circ \beta \circ \delta$ under D. □

Algorithm 1 Finite observational symbol discovery

Require: finite typed operation set $\mathcal{O}$, context suite $\mathfrak{C}$

1: **for all** $\alpha \in \mathcal{O}$ **do**
2: evaluate $\mathrm{Sig}_{\mathfrak{C}}(\alpha)$
3: **end for**
4: partition $\mathcal{O}$ by equality of signatures
5: return quotient classes and the induced composition table

Proposition 13.7 (Relative soundness and completeness)**.** *Algorithm 1 returns exactly the equivalence classes of $\sim_{\mathfrak{C}}$. Under contextual closure, its composition table is well defined.*

Proof. Two operations are placed in the same block exactly when their signatures agree. The preceding theorem supplies well-defined quotient composition. □

Remark 13.8. With noisy outputs, an ϵ-threshold relation need not be transitive. Practical discovery should therefore use confidence regions, complete-linkage constraints, or explicit congruence closure rather than naive pairwise thresholding.

Open theory boundary

The finite result is relative to a declared context suite. Discovering a minimal task-sufficient congruence under distribution shift, continuous operation families, and adaptive adversarial contexts remains open. The conjecture above is therefore narrowed, not proved in full.

14 Logical structure realized geometrically

14.1 Candidate geometries

Logical structure may be represented by geometry rather than explicit rule tables.

Logical relation	Candidate geometric realization
Equivalence	clusters or quotient spaces
Negation	involution $N^2 \approx I$
Implication	order cone
Conjunction	tensor or bilinear product
Temporal succession	semigroup flow
Reversible transformation	group action
Hierarchy	tree or hyperbolic geometry
Provenance	fiber indexed by source lineage
Fork	branching cospan
Shared meaning	common quotient or interlingua

14.2 Implication cones

Let $C \subseteq \mathcal{H}$ be a closed convex cone. One candidate semantics is

$$p \Rightarrow q \quad \Longleftrightarrow \quad z_q - z_p \in C.$$

This gives implication a partial-order geometry when $C \cap (-C) = \{0\}$.

14.3 Negation

A negation operator may satisfy

$$N^2 = I.$$

If propositions are represented as regions or subspaces, N should exchange a proposition with an appropriately defined contradictory region rather than merely multiply every vector by -1.

14.4 Conjunction

A higher-order conjunction may use a typed bilinear map

$$\mu_\wedge : \mathcal{H}_\varphi \otimes \mathcal{H}_\psi \to \mathcal{H}_{\varphi\wedge\psi}.$$

The tensor product records that conjunction may depend on interactions not reducible to a sum of independent features.

14.5 A finite ordered geometric fragment

Definition 14.1 (Order cone)**.** Let V be a real vector space and $C \subseteq V$ a convex cone. Define

$$x \preceq_C y \quad \Longleftrightarrow \quad y - x \in C.$$

The cone is *pointed* when $C \cap (-C) = \{0\}$.

Proposition 14.2 (Cone order)**.** *If C is a pointed convex cone, then $\preceq_C$ is a partial order.*

Proof. Reflexivity follows from $0 \in C$, transitivity from closure under addition, and antisymmetry from pointedness. □

Theorem 14.3 (Ordered involutive operator fragment)**.** *Let $C \subseteq V$ be a pointed convex cone. Suppose $N : V \to V$ is linear with*

$$N^2 = I, \qquad N(C) = -C,$$

and $\mu : V \otimes V \to V$ is bilinear, associative, commutative, has unit e, and is positive in the sense that

$$\mu(C, C) \subseteq C.$$

Then $(V, \preceq_C, \mu, e, N)$ is an ordered involutive commutative monoid: N reverses the order and μ is monotone in each argument on C.

Proof. The cone proposition gives the order. If $x \preceq_C y$, then $y - x \in C$, so

$$N(x) - N(y) = -N(y - x) \in C,$$

hence $N(y) \preceq_C N(x)$. Positivity and bilinearity imply

$$\mu(y, z) - \mu(x, z) = \mu(y - x, z) \in C$$

for $z \in C$, and similarly in the second argument. The monoid laws are assumed. □

Remark 14.4. This theorem realizes a useful logical fragment geometrically, but it does not imply Boolean idempotence, excluded middle, complement laws, or distributivity. Those laws must be declared separately and tested against the intended probabilistic, paraconsistent, or resource-sensitive semantics.

15 Probabilistic and paraconsistent semantics

15.1 Four-valued support

For a proposition p, store

$$v(p) = (s(p), r(p)) \in [0, 1]^2,$$

where s is support and r is refutation.

Region	Interpretation	Default policy
high s, low r	provisionally accepted	permit authorized use
low s, high r	provisionally rejected	do not use
high s, high r	conflicted	quarantine
low s, low r	unresolved	seek evidence or abstain

15.2 No explosion

The logic should not validate

$$p, \neg p \vdash q$$

for arbitrary q. Conflicting autobiographical or communicated content must remain localized.

A concrete four-valued algebra uses

$$\neg(s, r) = (r, s),$$

$$(s_1, r_1) \wedge (s_2, r_2) = (\min\{s_1, s_2\}, \max\{r_1, r_2\}),$$

$$(s_1, r_1) \vee (s_2, r_2) = (\max\{s_1, s_2\}, \min\{r_1, r_2\}).$$

A contradiction changes the value of formulas that actually depend on it; it does not assign arbitrary values to unrelated propositions.

15.3 Quantitative semantic stability

Let Θ denote the tuple of operator interpretations and let $d(\Theta, \Theta')$ measure representation error. Assume atomic valuations are Lipschitz, the selected connectives are nonexpansive under the sup metric, and every modal interpretation M is L_M-Lipschitz.

Theorem 15.1 (Structural stability of graded formulas)**.** *For every formula φ there exists a recursively computable constant L_φ such that*

$$|v_\Theta(\varphi, s) - v_{\Theta'}(\varphi, s)| \leq L_\varphi \, d(\Theta, \Theta').$$

One may take

$$L_{\neg\varphi} = L_\varphi, \qquad L_{\varphi\wedge\psi} = \max\{L_\varphi, L_\psi\}, \qquad L_{M\varphi} = L_M L_\varphi,$$

with analogous rules for the remaining nonexpansive connectives.

Proof. Proceed by structural induction. The atomic case is the Lipschitz assumption. Nonexpansiveness proves the connective steps. The modal step follows by composing Lipschitz maps. □

Corollary 15.2 (Threshold robustness)**.** *If a decision threshold θ has margin*

$$|v_\Theta(\varphi, s) - \theta| > L_\varphi d(\Theta, \Theta'),$$

then the thresholded decision is unchanged under replacement of Θ by Θ'.

15.4 Confidence as a first-class term

Endorsement may be represented by

$$K_a^q \varphi, \qquad q \in [0, 1].$$

The confidence value should be calibrated against observed success and tied to a declared evidence context.

15.5 Decision rules

A risk-sensitive action rule may require

$$J_a\varphi \wedge K_a^q\varphi \wedge q \geq \theta \wedge \mathrm{Auth}_a(\varphi) \to U_a\varphi.$$

The threshold θ depends on action cost and reversibility.

16 Provenance and proof-carrying execution

16.1 Provenance records

A content packet is

$$e = (c, a_{\text{src}}, L_{\text{src}}, \kappa, t, q, \mathcal{A}, d),$$

where c is content, a_{src} source agent, L_{src} source lineage, κ origin type, t time, q confidence, $\mathcal{A}$ permitted uses, and d an integrity digest.

16.2 Content is not ownership

Theorem 16.1 (Content-only ownership impossibility)**.** *Let two events e_1, e_2 have identical accessible content $C(e_1) = C(e_2) = c$, but opposite ownership labels. No classifier $f(c)$ depending only on content can classify both correctly.*

Proof. Because the input is identical, $f(C(e_1)) = f(c) = f(C(e_2))$, while the correct outputs differ. □

Corollary 16.2 (Equiprobable ownership lower bound)**.** *If the two provenance cases are equiprobable, every content-only classifier has error at least $1/2$ on the pair.*

16.3 Proof-gated actions

Let $\text{Pre}(\alpha)$ denote the logical precondition of action α. An executor accepts (α, p) only when the verifier establishes

$$p : \text{Pre}(\alpha).$$

Assumption 16.3 (Verifier soundness)**.** Whenever the verifier accepts $p : \varphi$, the proposition φ is true in the current operational model.

Theorem 16.4 (Proof-gated execution safety)**.** *Under verifier soundness, every executed action α satisfies its declared precondition* $\text{Pre}(\alpha)$*.*

Proof. The executor runs α only after acceptance of a proof term $p : \text{Pre}(\alpha)$. Soundness implies the precondition holds at execution time. □

Remark 16.5. A cryptographic digest proves byte integrity, not semantic authorization. The verifier must check lineage, provenance, conflict status, and action-specific constraints.

16.4 Linear provenance resources

A direct observation token $\text{Obs}_A(c)$ should not be freely duplicable across agents. Communication may use the rule

$$\text{Obs}_A(c) \multimap (\text{Access}_B(c) \otimes \text{Origin}_A(c)),$$

not

$$\text{Obs}_A(c) \multimap \text{Obs}_B(c).$$

This preserves the distinction between access and personal observation.

16.5 Proof transport between calculi

Let $\mathcal{L}_A$ and $\mathcal{L}_B$ be typed proof calculi. A translation Θ maps source types, formulas, contexts, and proof constructors into the target calculus.

Theorem 16.6 (Syntactic proof transport)**.** *Assume:*

1. *every source axiom φ maps to a target theorem $\Theta(\varphi)$;*
2. *for every source inference rule, the translated conclusion is derivable from the translated premises in the target calculus;*
3. *translation preserves typing and linear-resource usage.*

Then

$$\Gamma; \Pi \vdash_A \varphi \quad \Longrightarrow \quad \Theta(\Gamma); \Theta(\Pi) \vdash_B \Theta(\varphi).$$

Proof. Induct on the source derivation. Axioms are covered by the first assumption. Each inference step transports by the second assumption, while the third preserves well-formedness and resource accounting. □

Remark 16.7. This theorem transports derivability, not trust. Agent B must still trust the translation, the target verifier, and any external evidence referenced by the proof. Approximate neural proof objects require an additional certificate or re-verification layer.

17 Shared operator-symbolic languages

17.1 Common interlingua

Let $\mathcal{Z}_\star$ be a shared operator-symbolic space. Agent encoders and decoders are

$$E_A : \mathcal{H}_A \to \mathcal{Z}_\star, \qquad D_A : \mathcal{Z}_\star \to \mathcal{H}_A.$$

Cross-agent transport is

$$T_{A\to B} = D_B E_A.$$

17.2 Shared packet

A shared packet is

$$p = (c, o, L, \kappa, q, \mathcal{A}, t_{\exp}, d).$$

It contains content, origin, source lineage, provenance class, confidence, permitted uses, expiration, and digest.

17.3 Functional transport objective

A transport map should minimize

$$\begin{aligned}\mathcal{L} = {} & \lambda_1 \mathcal{L}_{\text{task}} + \lambda_2 \mathcal{L}_{\text{recon}} + \lambda_3 \mathcal{L}_{\text{func}} \\ & + \lambda_4 \mathcal{L}_{\text{cycle}} + \lambda_5 \mathcal{L}_{\text{causal}} + \lambda_6 \mathcal{L}_{\text{prov}} + \lambda_7 \mathcal{L}_{\text{capacity}}.\end{aligned}$$

17.4 Interlingua factorization

Proposition 17.1 (Exact shared factorization)**.** *Suppose $E_A : \mathcal{H}_A \to \mathcal{Z}_\star$ and $D_B : \mathcal{Z}_\star \to \mathcal{H}_B$ satisfy*

$$E_A \rho_A(g) = \rho_\star(g) E_A, \qquad D_B \rho_\star(g) = \rho_B(g) D_B.$$

Then $T_{A\to B} = D_B E_A$ is an intertwiner:

$$T_{A\to B} \rho_A(g) = \rho_B(g) T_{A\to B}.$$

Proof.

$$D_B E_A \rho_A(g) = D_B \rho_\star(g) E_A = \rho_B(g) D_B E_A.$$

□

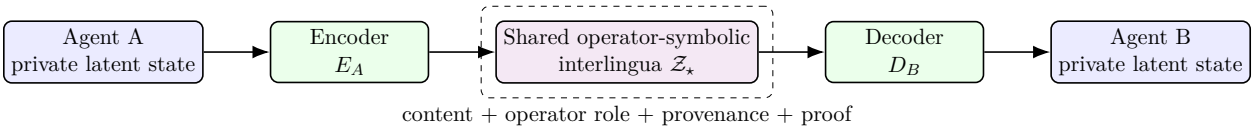


Figure 5: Heterogeneous agents communicate through a shared algebraic interlingua rather than by copying raw hidden states.

17.5 Communication is not merger

The agents remain distinct when private states, lineages, and independent control loops remain causally meaningful. A shared interlingua becomes a candidate group-level cognitive substrate only when critical state and recurrence exist irreducibly in the shared space.

17.6 Interface-decomposable communication

Let the joint deterministic state be $(x_A, x_B, w) \in X_A \times X_B \times W$. The system is *interface decomposable* when

$$\begin{aligned} x'_A &= F_A(x_A, w), \\ x'_B &= F_B(x_B, w), \\ w' &= G(w, m_A(x_A), m_B(x_B)), \end{aligned}$$

so each agent depends on the other only through the declared shared interface.

Theorem 17.2 (Private closure under a channel cut)**.** *Let $C_A \subseteq X_A$ and $C_B \subseteq X_B$ be private invariant sets. Suppose a channel cut replaces w by a fixed safe value w_0, and*

$$F_A(C_A, w_0) \subseteq C_A, \qquad F_B(C_B, w_0) \subseteq C_B.$$

Then after the cut, each private system remains in its own invariant set for every future step.

Proof. Apply the two inclusion assumptions inductively over time. □

This supplies a sufficient operational criterion for communication between persistent agents: the shared interface may improve performance, but removing it does not destroy each private invariant process.

17.7 Group-level task irreducibility

Let Y be a task variable. If

$$H(Y \mid X_A) > 0, \qquad H(Y \mid X_B) > 0, \qquad H(Y \mid X_A, X_B, W) = 0,$$

then neither private state alone determines the correct task output, while the joint state does.

Theorem 17.3 (Necessity of a joint representation)**.** *Under the entropy conditions above, no deterministic policy using only X_A or only X_B can solve every instance perfectly. A joint policy using (X_A, X_B, W) can in principle do so.*

Proof. Positive conditional entropy means at least two task outcomes remain possible after conditioning on the private state, so no deterministic private policy is always correct. Zero joint conditional entropy means Y is a function of the joint state. □

Proposition 17.4 (Task-conditioned communication lower bound)**.** *Let Y take values in a finite set $\mathcal{Y}$, let X_B be Agent B's private state, and let M be a message used to estimate Y with error probability P_e. Then*

$$I(Y; M \mid X_B) \geq H(Y \mid X_B) - h_2(P_e) - P_e \log_2(|\mathcal{Y}| - 1),$$

where h_2 is binary entropy.

Proof. By conditional Fano inequality,

$$H(Y \mid M, X_B) \leq h_2(P_e) + P_e \log_2(|\mathcal{Y}| - 1).$$

Subtract this bound from

$$I(Y; M \mid X_B) = H(Y \mid X_B) - H(Y \mid M, X_B).$$

□

The bound is representation independent. Any text, vector, packet, or operator-symbolic channel achieving the same error must transmit the required conditional task information. The open problem is to characterize the additional rate needed to preserve operator relations, provenance, and proof obligations simultaneously.

Open theory boundary

The results separate private persistence from group-level task irreducibility, but they do not provide a necessary-and-sufficient criterion for cognitive merger. A complete criterion likely requires recurrent group closure, interventionally irreducible state, lineage policy, and a declared level of analysis.

17.8 Canonical metadata and covert capacity

Let C be declared content, S prohibited private information, and M public metadata.

Theorem 17.5 (Canonical metadata noninterference)**.** *If trusted infrastructure computes metadata deterministically as*

$$M = g(C),$$

then

$$I(S; M \mid C) = 0.$$

Proof. Conditioned on C, the value of M is fixed and therefore independent of S. □

Proposition 17.6 (Finite metadata capacity)**.** *If, for each declared content value c, metadata belongs to a finite admissible set $\mathcal{M}_c$, then*

$$I(S; M \mid C = c) \le H(M \mid C = c) \le \log_2 |\mathcal{M}_c|.$$

For a sequence of n packets, capacity is at most the sum of the per-packet logarithmic bounds.

This does not prove the semantic channel is safe: agents may encode information in the declared operator choice itself. It does show why canonicalization, fixed precision, trusted timestamps, and finite codebooks remove whole classes of metadata channels.

18 Komorebi-style operator grammar

18.1 Analysis, packets, transfer, synthesis

A generic operator expression has the form

$$x \xrightarrow{\mathcal{A}} (P_\alpha x)_\alpha \xrightarrow{T_{\beta\alpha}} (P_\beta T P_\alpha x)_{\beta,\alpha} \xrightarrow{\mathcal{S}} x'.$$

The stages are:

1. analyze a state into packets;
2. select causally or semantically relevant packets;
3. transport packet relations;
4. validate provenance and residuals;
5. synthesize an updated state.

18.2 Exact packet block algebra

Let $\{P_\alpha\}_{\alpha=1}^m$ be pairwise orthogonal projectors satisfying

$$\sum_{\alpha=1}^{m} P_\alpha = I.$$

For any linear operator T, define transfer blocks

$$T_{\beta\alpha} = P_\beta T P_\alpha.$$

Theorem 18.1 (Packet block Pythagorean identity)**.** *For the Hilbert–Schmidt norm,*

$$\|T\|_{\mathrm{HS}}^2 = \sum_{\alpha,\beta} \|P_\beta T P_\alpha\|_{\mathrm{HS}}^2.$$

Proof. The operators $P_\beta T P_\alpha$ have pairwise orthogonal Hilbert–Schmidt supports, and their sum is T. □

Theorem 18.2 (Off-block commutator identity)**.**

$$\sum_{\alpha=1}^{m} \|[T, P_\alpha]\|_{\mathrm{HS}}^2 = 2 \sum_{\alpha \neq \beta} \|P_\beta T P_\alpha\|_{\mathrm{HS}}^2.$$

Proof. For fixed α, the nonzero blocks of $[T, P_\alpha]$ are $P_\beta T P_\alpha$ for $\beta \neq \alpha$ and $-P_\alpha T P_\gamma$ for $\gamma \neq \alpha$. Their supports are orthogonal. Summing over α counts each off-diagonal transfer block twice. □

18.3 Residuals

For a proposed transport T, define packet residuals

$$r_{\beta\alpha} = P_\beta(Tx) - P_\beta T P_\alpha x.$$

Large residuals signal missing cross-packet structure, misalignment, or invalid decomposition.

18.4 Holonomy

For a cycle of transports

$$A \to B \to C \to A,$$

define

$$H_{ABC} = T_{C\to A} T_{B\to C} T_{A\to B}.$$

Semantic cycle drift is

$$\epsilon_{\mathrm{hol}}(c) = \|H_{ABC}c - c\|.$$

This quantity measures whether meaning changes after a round trip through several agents or representations.

18.5 Multiresolution representation

Packetwise compression should preserve low-energy but functionally protected symbolic components. The language therefore needs a protected-rank or protected-distortion policy indexed by operator role, not only global numerical energy.

19 Interaction-induced latent geometry

19.1 Transient activation geometry

With fixed weights θ, interaction history changes activations:

$$h_{t+1} = F_\theta(h_t, x_t, C_t).$$

A structured interaction can constrain the trajectory to a manifold or attractor

$$h_t \in \mathcal{M}_C$$

without updating θ. The context then functions as a temporary program.

19.2 Persistent weight geometry

When training updates occur,

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta \mathcal{L}(\theta_n; E_n),$$

repeated interaction constraints reshape the weights. If the environment implements an algebra $\mathcal{A}$, training may induce an approximate representation

$$\rho_\theta : \mathcal{A} \to \mathrm{End}(\mathcal{H}).$$

19.3 Training objective

A prototype objective is

$$\begin{aligned}\mathcal{L} = \mathcal{L}_{\text{task}} &+ \lambda_1 \|h(gx) - M_g h(x)\|^2 \\ &+ \lambda_2 \|M_{g_2 g_1} - M_{g_2} M_{g_1}\|^2 + \lambda_3 \|M_e - I\|^2 \\ &+ \lambda_4 \mathcal{L}_{\text{bottleneck}} + \lambda_5 \mathcal{L}_{\text{proof}} + \lambda_6 \mathcal{L}_{\text{prov}}.\end{aligned}$$

At low loss, M_g approximates the interaction algebra.

19.4 From interaction relations to an internal representation

Let

$$\mathcal{A} = \mathbb{R}\langle \mathcal{G} \rangle / \langle \mathcal{R} \rangle$$

be an algebra presented by generators $\mathcal{G}$ and relations $\mathcal{R}$.

Theorem 19.1 (Presentation-to-representation extension)**.** *Suppose matrices* $\{M_g : g \in \mathcal{G}\} \subseteq \mathrm{End}(\mathcal{H})$ *satisfy*

$$r(M_g : g \in \mathcal{G}) = 0$$

for every defining relation $r \in \mathcal{R}$*. Then the generator assignment extends uniquely to an algebra homomorphism*

$$\rho : \mathcal{A} \to \mathrm{End}(\mathcal{H}).$$

Proof. The generator assignment extends uniquely to the free algebra $\mathbb{R}\langle \mathcal{G} \rangle$. Because every defining relation lies in its kernel, the map factors uniquely through the quotient by $\langle \mathcal{R} \rangle$. □

This result formalizes the strongest version of interaction-induced geometry: an external protocol specifies generators and relations, while training searches for an internal realization of that presented algebra. The realization is generally identifiable only up to gauge and quotient.

Conjecture 19.2 (Context-to-weight consolidation)**.** *A repeatedly used activation-level operator scheme can be consolidated into persistent weight geometry through replay, distillation, fine-tuning, or reinforcement, provided the update process preserves the interaction algebra and penalizes simpler shortcut solutions.*

20 Tensor and nonlinear generalizations

20.1 Tensor-valued states

An agent state may be

$$X_t \in \mathcal{X}_{\text{agent}} \otimes \mathcal{X}_{\text{time}} \otimes \mathcal{X}_{\text{modality}} \otimes \mathcal{X}_{\text{memory}} \otimes \mathcal{X}_{\text{causal}}.$$

Higher-order relations may require multilinear maps

$$\mathcal{T} : \mathcal{X}_1 \otimes \cdots \otimes \mathcal{X}_k \to \mathcal{Y}.$$

Theorem 20.1 (Tensor products preserve semantic intertwiners)**.** *For* $i = 1, 2$*, let* $T_i : \mathcal{H}_{A_i} \to \mathcal{H}_{B_i}$ *intertwine representations* ρ_{A_i} *and* ρ_{B_i}*. Then*

$$T_1 \otimes T_2$$

intertwines the tensor-product representations:

$$(T_1 \otimes T_2)(\rho_{A_1}(g_1) \otimes \rho_{A_2}(g_2)) = (\rho_{B_1}(g_1) \otimes \rho_{B_2}(g_2))(T_1 \otimes T_2).$$

Proof. Use the mixed-product identity for tensor products and the two intertwining equations, then extend linearly. □

20.2 Nonlinear retractions

Replace a linear projector P by an idempotent nonlinear retraction

$$\Pi \circ \Pi = \Pi.$$

Its image may be a curved manifold rather than a subspace.

20.3 Nonlinear semantics

Let

$$G_\theta : \mathcal{X} \to \mathcal{H}, \qquad R_\phi : \mathcal{H} \to \mathcal{X}.$$

Binding residuals use a metric or divergence:

$$\epsilon_{\text{bind}} = d_{\mathcal{H}}(\Pi_J(h), G_\theta(\Pi_S(x))).$$

20.4 Coordinate covariance

For diffeomorphisms $\phi : \mathcal{X} \to \mathcal{X}'$ and $\psi : \mathcal{H} \to \mathcal{H}'$, transport

$$\Pi'_S = \phi \circ \Pi_S \circ \phi^{-1}, \qquad G' = \psi \circ G \circ \phi^{-1}.$$

Meaning should be defined through commuting diagrams and causal behavior, not fixed coordinates.

20.5 Koopman and kernel lifts

Nonlinear dynamics

$$x_{t+1} = F(x_t)$$

may be lifted to a space of observables where a Koopman operator acts linearly:

$$(\mathcal{K}g)(x) = g(F(x)).$$

Kernel embeddings provide another route to approximately linear packet geometry.

20.6 Finite-horizon approximate conjugacy

Let (X_A, d_A) and (X_B, d_B) be metric spaces with dynamics F_A and F_B, and let $T : X_A \to X_B$. Assume

$$d_B(TF_A(x), F_BT(x)) \le \epsilon$$

for all relevant x, and suppose F_B is L-Lipschitz.

Theorem 20.2 (Nonlinear conjugacy error propagation)**.** *For every $n \ge 1$,*

$$d_B\left(TF_A^n(x), F_B^nT(x)\right) \le \epsilon \sum_{k=0}^{n-1} L^k.$$

Hence the error is at most $n\epsilon$ when $L = 1$, and at most $\epsilon/(1-L)$ uniformly in n when $L < 1$.

Proof. Let $e_n = d_B(TF_A^n(x), F_B^n T(x))$. The one-step defect and Lipschitz property give

$$e_{n+1} \leq \epsilon + Le_n.$$

Iterating from $e_0 = 0$ proves the bound. □

Corollary 20.3 (Exact nonlinear semantic transport)**.** *If $\epsilon = 0$, then $TF_A^n = F_B^n T$ for every finite horizon n.*

Open theory boundary

The theorem controls one deterministic dynamics map. A general theory of nonlinear operator-symbolic equivalence still needs action-indexed approximate bisimulation, stochastic kernels, partial observations, open-set symbol creation, and semantic stability under changing state-dependent charts.

21 A compiler architecture

21.1 Front end

Inputs may be text, images, sensor streams, actions, or internal activations. Learned encoders convert them into latent states and candidate operator relations.

21.2 Intermediate representation

The operator-symbolic intermediate representation contains the lowered form of OSL-Core and the richer $\mathbb{L}_{\mathrm{AI}}$ family:

- typed states and packets;
- operator graphs;
- provenance and lineage;
- uncertainty and conflict values;
- proof obligations;
- expected causal consequences.

A compact static-single-assignment style record is:

Operator-symbolic expression

```
%r7 : Workspace = transport %r3 via T_A_to_B
      [origin=B, lineage=L_B, confidence=.81]
%p9 : Proof = verify Origin(%r7,B) and Authorized(%r7,planning)
%a2 : Action = alternate_route(%r7)
      requires %p9 effects {externalAct}
```

Every instruction therefore carries a type, provenance context, proof obligations, and an effect summary.

21.3 Verifier

The verifier checks:

- type correctness;
- operator-domain compatibility;
- provenance constraints;
- proof validity;
- conflict and quarantine status;
- action authorization;
- resource and safety bounds.

21.4 Effect confinement

Assume every primitive transition emits only effects declared by its typing rule, and composite typing takes unions over sequential and conditional branches.

Theorem 21.1 (Effect confinement)**.** *If*

$$\Gamma; \Pi \vdash \alpha : \tau \rightsquigarrow \sigma \,!\, \mathcal{E},$$

then every finite execution trace of α emits only effects in $\mathcal{E}$, provided the operational semantics preserves typing.

Proof. Induct on the execution derivation. Primitive steps satisfy the claim by assumption. Sequential and conditional steps inherit the union of the effects of their subderivations. Typing preservation keeps the declared effect set valid after each step. □

21.5 Back end

The back end renders the verified representation into:

- natural-language output;
- code;
- plans;
- tool calls;
- motor actions;
- memory writes.

22 Assumption ledger

Algorithm 2 Operator-symbolic execution cycle

Require: observation o_t, latent state h_t, memory M_t

$z_t \leftarrow \text{Encode}(o_t, h_t, M_t)$
$\mathcal{P}_t \leftarrow \text{Packetize}(z_t)$
$\mathcal{O}_t \leftarrow \text{InferOperators}(\mathcal{P}_t)$
$\Phi_t \leftarrow \text{ConstructFormulas}(\mathcal{P}_t, \mathcal{O}_t)$
$\Pi_t \leftarrow \text{ProveObligations}(\Phi_t)$
$(\alpha_t, p_t) \leftarrow \text{Plan}(\Phi_t, \Pi_t)$
if $\text{Verify}(p_t : \text{Pre}(\alpha_t))$ **then**
    execute α_t
    observe consequence y_{t+1}
    update geometry, operators, beliefs, proofs, and memory
else
    quarantine proposal and request more evidence
end if

Result family	Assumptions	What is not claimed
Intertwiner theorems	finite-dimensional linear realizations; declared operator algebra	discovery of the correct algebra in trained models
Observational congruence	finite operation and context suites; exact observed equality	robustness to arbitrary distribution shift
Geometric logic	pointed cone; explicit involution and positive bilinear product	complete Boolean or intuitionistic semantics
Graded stability	Lipschitz atomic and modal interpretations	calibrated probabilities without empirical fitting
Proof safety	sound verifier and correct specification	safety against misspecified preconditions
Shared-language results	declared encoders, decoders, provenance fields, and finite targets	a universal criterion for cognitive merger
Coalgebraic trace transport	deterministic action-indexed transition systems	stochastic or partially observed equivalence
Packet identities	complete orthogonal projector family	optimality of a learned packet decomposition
Nonlinear transport	deterministic map; uniform one-step defect; Lipschitz target dynamics	open-ended stochastic bisimulation
Effect confinement	sound effect typing and preservation	absence of semantic covert channels inside allowed effects

23 Formal theorem program

23.1 Results established here

The expanded finite theory establishes:

1. exact intertwiners preserve all polynomial operator expressions;
2. approximate intertwiners obey product and presented-relation bounds;
3. operator meaning is invariant under change of coordinates;
4. finite contextual test suites recover observational congruences under closure;
5. operator congruences induce quotient symbol algebras;
6. pointed cones, involutions, and positive bilinear products realize an ordered logical fragment;
7. graded formula semantics is stable under bounded interpretation error;
8. content alone cannot determine ownership when provenance labels differ;
9. proof-gated execution is safe under verifier soundness;
10. proofs translate between calculi when axioms, rules, types, and resources are preserved;
11. exact shared interlingua factorizations produce cross-agent intertwiners;
12. coalgebra homomorphisms preserve every finite action-conditioned observation trace;
13. private invariant sets survive declared channel cuts under fallback closure;
14. group-level task information can be irreducible to either private state;
15. conditional Fano bounds lower-bound task-relevant shared-language bandwidth;
16. canonical metadata eliminates conditional metadata leakage, and finite alphabets bound capacity;
17. packet blocks satisfy exact Hilbert–Schmidt Pythagorean and commutator identities;
18. presented interaction algebras extend uniquely to internal representations;
19. tensor products preserve semantic intertwiners;
20. nonlinear approximate conjugacy has an explicit finite-horizon error bound;
21. verified programs satisfy effect confinement under a sound effect system.

23.2 Remaining theorem targets

Open Problem 23.1 (Distribution-shift congruence recovery)**.** Under what conditions does training recover the minimal task-sufficient operator congruence when contexts are continuous, adaptive, and outside the original support?

Open Problem 23.2 (Approximate proof transport)**.** How should neural or probabilistic proof certificates be translated between independently trusted verifiers while retaining quantitative soundness guarantees?

Open Problem 23.3 (Necessary and sufficient merger criterion)**.** Can one characterize the transition from communication to an irreducible distributed agent using causal factorization, recurrent closure, lineage, and interventionally necessary shared state?

Open Problem 23.4 (Shared-language rate–distortion)**.** Characterize the minimum dimension and bandwidth of an interlingua that preserves a declared operator algebra, provenance policy, and task loss within tolerances.

Open Problem 23.5 (Semantic covert channels)**.** Bound undeclared information carried by legal operator choice, proof selection, packet sequencing, timing, and continuous latent coefficients after metadata canonicalization.

Open Problem 23.6 (Real-model identification)**.** Determine whether trained open-weight models contain stable operator presentations, congruence classes, proof-like states, and shared intertwiners that survive causal intervention and strong alternative baselines.

24 Experimental program

24.1 Experiment A: operator algebra induction

Train models on transformations generated by a finite algebra. Hold out unseen compositions. Test whether

$$M_{g_3 g_2 g_1} \approx M_{g_3} M_{g_2} M_{g_1}$$

and whether the relation survives random basis changes.

24.2 Experiment B: symbol emergence

Create multiple concrete operations that are equivalent under task consequences. Test whether the model compresses them into a shared latent class and transfers the symbol to new domains.

24.3 Experiment C: heterogeneous-agent communication

Give two models different architectures and complementary private observations. Restrict communication to a shared operator-symbolic interlingua. Compare against text, arbitrary vectors, cross-attention, and no-channel controls.

24.4 Experiment D: provenance-sensitive communication

Transmit identical content through observation, inference, foreign communication, simulation, and injection. Measure ownership, memory write, planner use, and quarantine.

24.5 Experiment E: proof-gated planning

Compare unrestricted planning with proof-carrying planning under hidden provenance conflicts and adversarially rehashed receipts.

24.6 Experiment F: context-to-weight consolidation

First induce an operator scheme in context with frozen weights. Then distill or fine-tune on successful traces. Test whether the scheme persists without the original context and whether it remains gauge-equivalent.

24.7 Experiment G: nonlinear transfer

Use nonlinear coordinate changes, component substitutions, and task-family shifts. Compare linear intertwiners, nonlinear flows, Koopman lifts, and kernel methods.

24.8 Evaluation standards

A decisive benchmark should separate discovery, transport, proof, and control. At minimum it should include:

- held-out operator compositions and held-out task families;
- random and adversarial coordinate changes;
- same-content/different-provenance cases;
- independently initialized and architecturally heterogeneous agents;
- strong latent baselines: raw vectors, PCA, sparse features, canonical correlation, and supervised probes;
- proof-free, provenance-free, and planner-binding ablations;
- bandwidth-matched text and vector channels;
- post-hoc leakage audits and paired-execution hyperproperty tests;
- predeclared thresholds and untouched model–task holdouts.

Primary metrics should include unseen-composition accuracy, relation residual, proof acceptance precision, unauthorized-action rate, provenance error, transport rate–distortion, semantic holonomy, calibration, and causal task regret.

24.9 Essential ablations

Ablation	Expected failure
Remove operator composition loss	weak unseen-composition transfer
Randomize transport map	semantic corruption across agents
Remove provenance	ownership and memory contamination
Remove proof verifier	unauthorized actions increase
Remove latent layer	brittle perception and novelty handling
Remove symbolic layer	poor auditability and provenance control
Remove feedback	no consequence-sensitive semantic revision
Increase channel capacity without structure	uninspectable state dumping

25 Falsification criteria

The theory is weakened if any of the following holds robustly across strong benchmarks:

1. raw latent vectors transfer as reliably as operator-preserving interlinguas under unseen model pairs;
2. symbolic proof and provenance provide no measurable safety or audit benefit;

3. learned operator relations fail to generalize beyond trained compositions;
4. operator geometry is not stable under coordinate changes;
5. content-only systems solve same-content/different-origin tasks as well as provenance-aware systems;
6. quotient symbols do not improve compression or transfer;
7. simpler memory-and-routing systems explain every predicted dissociation;
8. the shared language cannot be distinguished from an arbitrary high-bandwidth code;
9. proof-gated systems do not reduce unauthorized effects under matched task performance;
10. finite symbol classes fail immediately under small unseen context changes;
11. nonlinear transport bounds do not predict observed semantic degradation;
12. group-level effects disappear after controlling for simple information pooling;
13. the proposed intermediate representation offers no compression, transfer, verification, or causal-control advantage over simpler typed message passing.

26 Limitations and boundaries

26.1 No unique internal language

Behavior generally determines an internal representation only up to gauge, quotient, or nonlinear reparameterization. The theory therefore predicts an equivalence class of languages rather than one canonical coordinate system.

26.2 Interpretability remains incomplete

A typed operator symbol may be easier to inspect than a raw activation, but its learned semantics may still be distributed, approximate, and context dependent.

26.3 Proof verifier assumptions

Proof-gated safety depends on verifier soundness and the completeness of declared preconditions. A verifier can certify the wrong specification perfectly.

26.4 Optimization can find shortcuts

Training may exploit accidental metadata, labels, or benchmark structure instead of learning the intended algebra. External holdouts and causal interventions are required.

26.5 Natural language remains useful

The proposed language is an intermediate representation, not a replacement for human communication. Natural language remains essential for collaboration, explanation, and governance.

26.6 No consciousness claim

The existence of an internal language of operators and logic would not establish awareness or subjective experience.

27 Discussion

27.1 Language as transformation algebra

The deepest form of the proposal is that a machine language need not fundamentally be a dictionary. It can be an algebra of transformations. A symbol is meaningful because it occupies a stable operator role. A proposition constrains admissible states. A proof certifies a transformation path. A memory has meaning through provenance and predictive use. An action has meaning through its causal consequences.

27.2 A cognitive intermediate representation

Compiler intermediate representations allow heterogeneous programming languages and hardware targets to share a structured middle layer. An operator-symbolic language could play a similar role for cognition:

$$\begin{aligned}\text{heterogeneous perception and latent models} &\to \mathbb{L}_{\mathrm{AI}} \\ &\to \text{heterogeneous planners, tools, and actuators.}\end{aligned}$$

27.3 Machine-native communication

Different models need not share vectors or vocabularies. They need compatible realizations of the same abstract transformation algebra, plus transports that preserve operator roles, provenance, uncertainty, and proof obligations.

27.4 Relation to persistent agency

When combined with temporal lineage, self-binding, and recurrent consequence updates, the operator-symbolic language becomes a candidate formal language for persistent agents. The present paper focuses on the language layer; a fuller theory of persistent computational agency requires closure across memory, identity, planning, and feedback.

27.5 What the theory predicts about learned representations

If an interaction protocol repeatedly rewards a compositional algebra, then successful models should exhibit approximately relation-preserving operator families, not merely clusters of output labels. The geometry may differ across models, but causal interventions should reveal the same operator equations up to transport. This predicts a concrete interpretability target: search for generators, relations, quotient classes, and intertwining maps rather than one privileged vocabulary direction.

27.6 Why operators and logic are complementary

Operators alone can transform states but do not determine which transformations are permitted, owned, or sufficiently justified. Logic alone can state admissibility but does not discover robust representations from noisy data. The proposed language therefore treats geometry as the carrier, operators as executable structure, and logic as proof-carrying governance.

28 Conclusion

We proposed an operator-symbolic language for artificial intelligence:

$$\boxed{\text{continuous geometry} + \text{typed operators} + \text{symbolic logic} + \text{proof and provenance.}}$$

The central principle is that different systems need not share coordinates. They must preserve the same transformation algebra up to transport and approximation. Symbols may emerge as compressed invariants of repeated operator patterns. Logic constrains which transformations, beliefs, memories, and actions are valid. Proof paths make reasoning executable and auditable. Provenance distinguishes access from ownership. Feedback grounds the language in causal consequences.

The strongest interpretation is therefore:

> An AI-native language is not merely a set of tokens. It is a typed, proof-carrying algebra of transformations over learned geometry, with causal consequences supplying semantics.

The finite core is now developed across exact, approximate, geometric, provenance, communication, nonlinear, compiler-level, and executable-language results. Version 0.3 retains the version 0.2 AST, proof, capability, contract, rollback, session, and neural-boundary layers and adds linear proof resources, append-only audit semantics, semantic statement boundaries, finite graph verification, and independent finite metatheory certificates. OSL-Core demonstrates that the proposal can be rendered as an actual source language with deterministic operator execution, typed provenance, proof-gated memory, capability-gated effects, canonical receipts, and expected rejection of invalid programs. The framework remains empirically unverified at scale, but it gives a precise research target for neural-symbolic reasoning, heterogeneous-agent communication, mechanistic interpretability, memory provenance, and persistent control.

A Notation

Symbol	Meaning
$\mathbb{L}_{\mathrm{AI}}$	operator-symbolic AI language
$\mathcal{H}_a$	private latent space of agent a
$\mathcal{Z}_\star$	shared operator-symbolic interlingua
$\mathcal{A}$	abstract operator algebra
ρ_a	realization of $\mathcal{A}$ in agent a
$T_{A\to B}$	transport from agent A to agent B
E_A, D_A	shared encoder and decoder
$J_a\varphi$	workspace accessibility
$B_{a,L}\varphi$	binding to agent a under lineage L
$O_a^\kappa\varphi$	autobiographical or provenance-typed ownership
$K_a^q\varphi$	epistemic endorsement with confidence q
$U_a\varphi$	causal use in planning
$Q_a\varphi$	quarantine
$[\![\cdot]\!]$	denotational semantics
$\epsilon_T(g)$	intertwining residual
$\gamma : S \to O \times S^{\mathsf{A}}$	action-indexed agent coalgebra
H_{ABC}	inter-agent holonomy operator

Symbol	Meaning
$\mathrm{Sig}_{\mathfrak{C}}(\alpha)$	finite observational signature of operation α
$\mathcal{E}$	declared effect set of a program
$\preceq_C$	partial order induced by cone C

B Typing rules

Representative typing judgments include:

$$\frac{\Gamma \vdash t : \tau \qquad g : \tau \to \sigma}{\Gamma \vdash g(t) : \sigma},$$

$$\frac{\Gamma \vdash \alpha : \tau \to \sigma \qquad \Gamma \vdash \beta : \sigma \to \upsilon}{\Gamma \vdash \alpha;\beta : \tau \to \upsilon},$$

$$\frac{\Gamma;\Pi \vdash p : \mathrm{Pre}(\alpha)}{\Gamma;\Pi \vdash \mathrm{Execute}(\alpha, p) : \mathrm{Result}(\alpha)}.$$

Effect composition is typed by

$$\frac{\Gamma;\Pi \vdash \alpha : \tau \rightsquigarrow \sigma \,!\, \mathcal{E}_1 \qquad \Gamma;\Pi \vdash \beta : \sigma \rightsquigarrow \upsilon \,!\, \mathcal{E}_2}{\Gamma;\Pi \vdash \alpha;\beta : \tau \rightsquigarrow \upsilon \,!\, (\mathcal{E}_1 \cup \mathcal{E}_2)}.$$

A communicated observation preserves origin:

$$\frac{\Gamma;\Pi \vdash o : \mathrm{Obs}_A(c)}{\Gamma;\Pi \vdash \mathrm{send}_{A\to B}(o) : \mathrm{Access}_B(c) \otimes \mathrm{Origin}_A(c)}.$$

There is intentionally no rule deriving $\mathrm{Obs}_B(c)$ from this conclusion.

C A minimal packet schema

```
OperatorSymbolicPacket(
    content,
    content_type,
    operator_role,
    source_agent,
    source_lineage,
    provenance_class,
    confidence,
    authorization_scope,
    proof_digest,
    predicted_consequence,
    expiry_step,
    packet_digest,
)
```

D Reference implementation modules

The v0.5 release contains a statically checked executable finite core, an independent receipt kernel, two complementary exhaustive finite transition-model layers, source-bound semantic trace certificates, and a proposed full implementation layout.

D.1 Executable finite kernel

```
reference/
|-- osl_model.py               # typed objects and proof resources
|-- osl_parser.py              # AST parser and static checker
|-- osl_core.py                # deterministic source runtime
|-- osl_invariants.py          # runtime and ledger invariants
|-- osl_kernel.py              # independent receipt/proof replay
|-- verify_trace.py            # source-bound deterministic replay
|-- osl_formal_kernel.py       # integrated exhaustive abstraction
|-- osl_metacore.py            # five decomposed finite models
|-- osl_session_verify.py      # finite protocol graph verifier
|-- osl_information_flow.py    # explicit-flow certificate
|-- generate_v03_evidence.py   # deterministic evidence builder
`-- verify_osl_core_v03.py     # unified finite certificate
examples/
|-- 01_direct_observation.osl
|-- ...
|-- 42_verified_three_agent_session.osl
|-- 91_forge_observation.osl
`-- 114_linear_ownership_reused.osl
tests/
|-- test_osl_core.py
|-- test_osl_core_v02.py
|-- test_osl_core_v03.py
|-- test_osl_metatheory.py
|-- test_osl_v03_certificates.py
`-- test_osl_certificates.py
```

D.2 Proposed full implementation

```
operator_symbolic_ai/
|-- syntax/
|   |-- lexer.py
|   |-- parser.py
|   `-- ast.py
|-- typing/
|   |-- kinds.py
|   |-- provenance_resources.py
|   `-- effects.py
|-- operators/
|   |-- linear.py
|   |-- nonlinear.py
|   |-- tensor.py
|   `-- transport.py
|-- logic/
|   |-- formulas.py
|   |-- paraconsistent.py
```

```
|   |-- temporal.py
|   `-- proof_terms.py
|-- runtime/
|   |-- packets.py
|   |-- verifier.py
|   |-- memory.py
|   |-- planner.py
|   `-- receipts.py
|-- compiler/
|   |-- lower.py
|   |-- osl_ir.py
|   |-- verify.py
|   `-- backends.py
`-- experiments/
```

E Theorem and dependency status

Result	Status	Principal dependency
Polynomial semantic preservation	proved	exact intertwiner
Approximate relation preservation	proved	bounded generator norms and residuals
Finite contextual congruence	proved	finite context closure
Ordered geometric fragment	proved	pointed cone, involution, positive product
Graded semantic stability	proved	Lipschitz interpretations
Ownership impossibility	proved	identical content, differing provenance labels
Proof-gated safety	proved	verifier soundness
Proof transport	proved	axiom and rule preservation
Private closure after channel cut	proved	invariant fallback dynamics
Group-level task irreducibility	proved	conditional entropy separation
Communication rate lower bound	proved	finite target and conditional Fano inequality
Coalgebraic trace preservation	proved	exact coalgebra homomorphism
Metadata noninterference and capacity	proved	trusted canonicalization or finite alphabet
Packet block identities	proved	complete orthogonal projectors
Presentation extension	proved	exact satisfaction of defining relations
Tensor intertwiner transport	proved	factorwise intertwiners
Nonlinear finite-horizon transport	proved	uniform defect and Lipschitz dynamics
Effect confinement	proved	sound effect typing and preservation
Finite proof-resource preservation	proved in finite core	bounded proof-resource transition
Receipt-kernel relative soundness	computer-assisted	independent finite receipt grammar

Result	Status	Principal dependency
Integrated abstract-kernel certificate	exhaustive finite	740-state fixed point
Decomposed metacore certificate	exhaustive finite	five fixed points, 1,590 states
Finite protocol completion	proved for verified graph	terminal reachability and no deadlock
Explicit-flow noninterference	proved for syntax fragment	no secret-to-metadata path
Open-world symbol discovery	open	distribution shift and adaptive contexts
Cognitive merger criterion	open	recurrent causal irreducibility
Real-model identification	empirical	internal access and intervention hooks

F Open theory modules

Open theory boundary

Mechanized metatheory. The release now includes exhaustive finite transition certificates and an independent receipt kernel. A proof-assistant formalization of the syntax, linear provenance resources, effect system, operational semantics, and kernel correspondence still remains to be constructed.

Open theory boundary

Approximate proof transport. Exact derivations transport under rule-preserving translations. Neural certificates, probabilistic evidence, independently rooted trust, and quantitative proof degradation require a richer proof-relevant metric semantics.

Open theory boundary

Open-world congruence discovery. Finite contextual signatures recover the quotient induced by a declared test suite. Minimal symbol discovery under continuous actions, adversarial contexts, and distribution shift remains open.

Open theory boundary

Communication versus merger. Private channel-cut closure and group task irreducibility are only sufficient operational tests. A necessary-and-sufficient criterion for a new distributed agent likely requires recurrent group closure, causal irreducibility, and a lineage semantics.

Open theory boundary

Real-model identification. It remains to determine whether trained neural models contain stable operator presentations, quotient symbols, proof-like states, and transport maps that survive causal intervention and outperform strong representation-learning baselines.

G Selected references and bibliographic audit boundary

The following references identify neighboring literatures and standard mathematical tools. Contemporary software and manuscript entries require archival normalization before submission.

References

[1] A. Newell. Physical symbol systems. *Cognitive Science*, 1980.

[2] P. Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 1990.

[3] J.-Y. Girard. Linear logic. *Theoretical Computer Science*, 1987.

[4] D. Harel, D. Kozen, and J. Tiuryn. *Dynamic Logic*. MIT Press, 2000.

[5] P. Gärdenfors. *Conceptual Spaces: The Geometry of Thought*. MIT Press, 2000.

[6] A. S. d'Avila Garcez, L. C. Lamb, and D. M. Gabbay. *Neural-Symbolic Cognitive Reasoning*. Springer, 2008.

[7] A. Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[8] J. von Oswald et al. Transformers learn in-context by gradient descent. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[9] F. W. Lawvere. Metric spaces, generalized logic, and closed categories. 1973. Bibliographic details to verify.

[10] N. D. Belnap. A useful four-valued logic. In *Modern Uses of Multiple-Valued Logic*, 1977.

[11] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[12] E. M. Clarke, O. Grumberg, and D. A. Peled. *Model Checking*. MIT Press, 1999.

[13] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[14] J. J. M. M. Rutten. Universal coalgebra: a theory of systems. *Theoretical Computer Science*, 249, 2000.

[15] T. M. Cover and J. A. Thomas. *Elements of Information Theory*, second edition. Wiley, 2006.

[16] S. Mac Lane. *Categories for the Working Mathematician*, second edition. Springer, 1998.

[17] B. Fong and D. I. Spivak. *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press, 2019.

[18] J. Pearl. *Causality*, second edition. Cambridge University Press, 2009.

[19] C. Baier and J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008.

[20] D. Sangiorgi and D. Walker. *The Pi-Calculus: A Theory of Mobile Processes*. Cambridge University Press, 2001.

[21] Komorebi P. Projector-resolved operator and transfer calculus. Versioned software and manuscript series. Permanent archival citation and theorem crosswalk pending.

[22] Komorebi P. Self-indexed workspace closure: operator geometry, provenance logic, and recurrent computational agency. Expanded symbolic theory draft, 2026.